Lecture 1.2: Geospatial Data Types and Python for GIS

CSE620B: Remote Sensing & Computer Vision

Dr. John C. Femiani
Miami University

Introduction to Python Geospatial Libraries

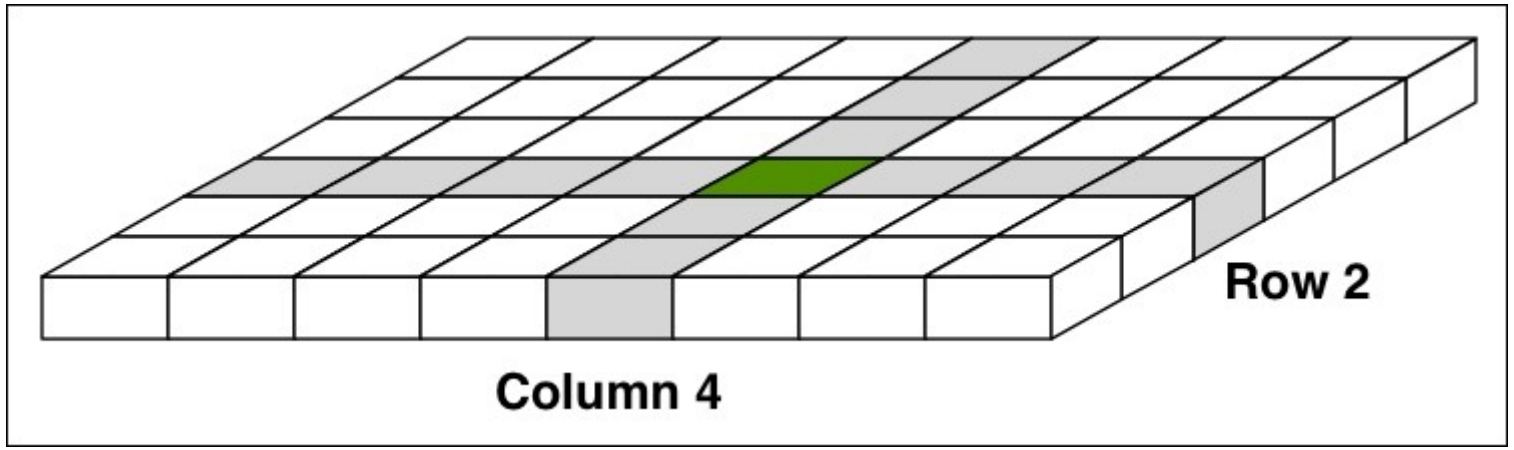
- **Python's Role in Geospatial Development**
  - Widely used due to its robust libraries and active community

- **Why Use Libraries Over Custom Code?**
  - Efficient, reliable, and well-tested solutions

- **Shift from Traditional Tools**
  - **Book Focus:** GDAL/OGR for geospatial data manipulation
  - **Modern Approach:** Emphasizing **`rasterio`** and **`shapely`** for their Pythonic interface and ease of use

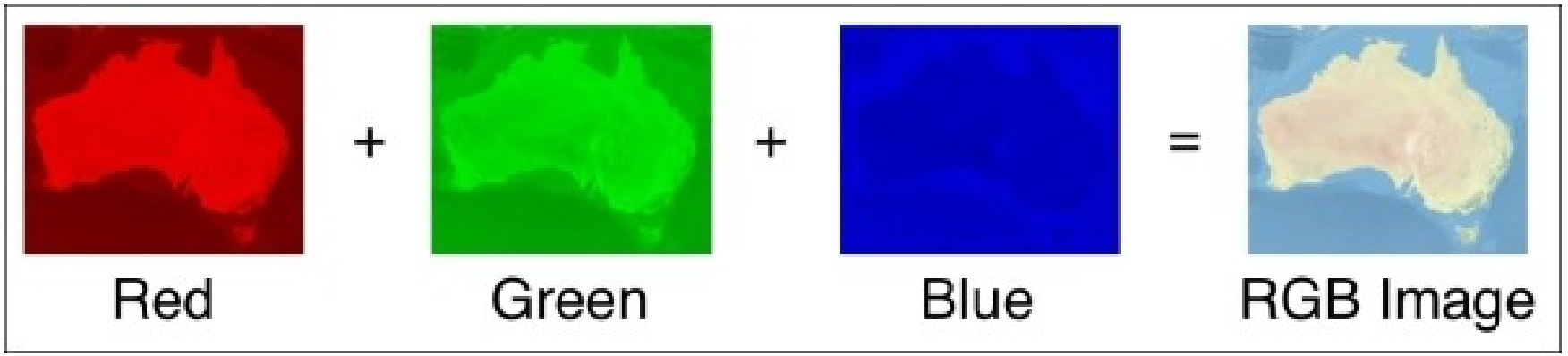
Recap of NumPy Arrays for Raster Data

- **Understanding Image as an Array:**
  - Each raster image is represented as a NumPy array
  - For multi-band images (e.g., RGB), each band is a separate array
- **Array Structure:**
  - 2D arrays for single-band images (rows x columns)
  - 3D arrays for multi-band images (bands x rows x columns)



Understanding Raster Bands

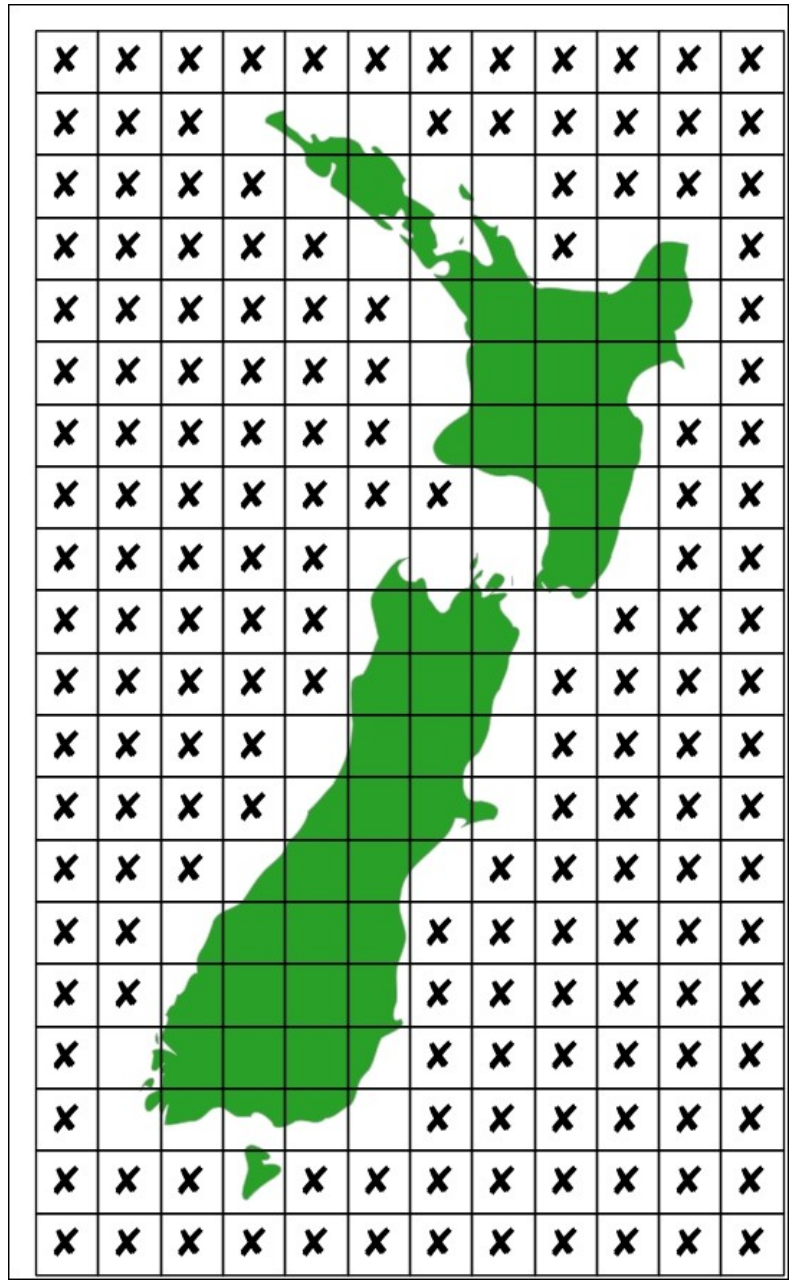
- **RGB Image Composition:**
  - Each band represents a single color component (Red, Green, Blue)
  - Bands are combined to create a full-color image
- **Working with Bands in `rasterio`:**
  - Easy access to individual bands as NumPy arrays
  - Example: Extracting and visualizing each band separately



Handling NoData Values

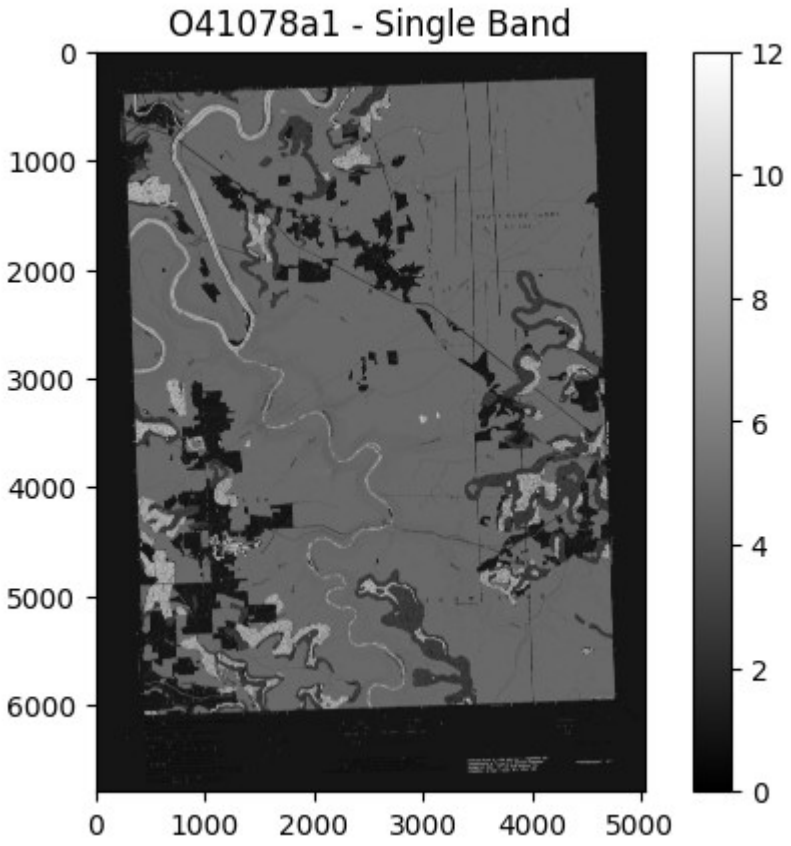
- What are NoData Values?
  - Cells with missing or undefined data, marked as NoData
  - Important for accurate analysis and avoiding errors

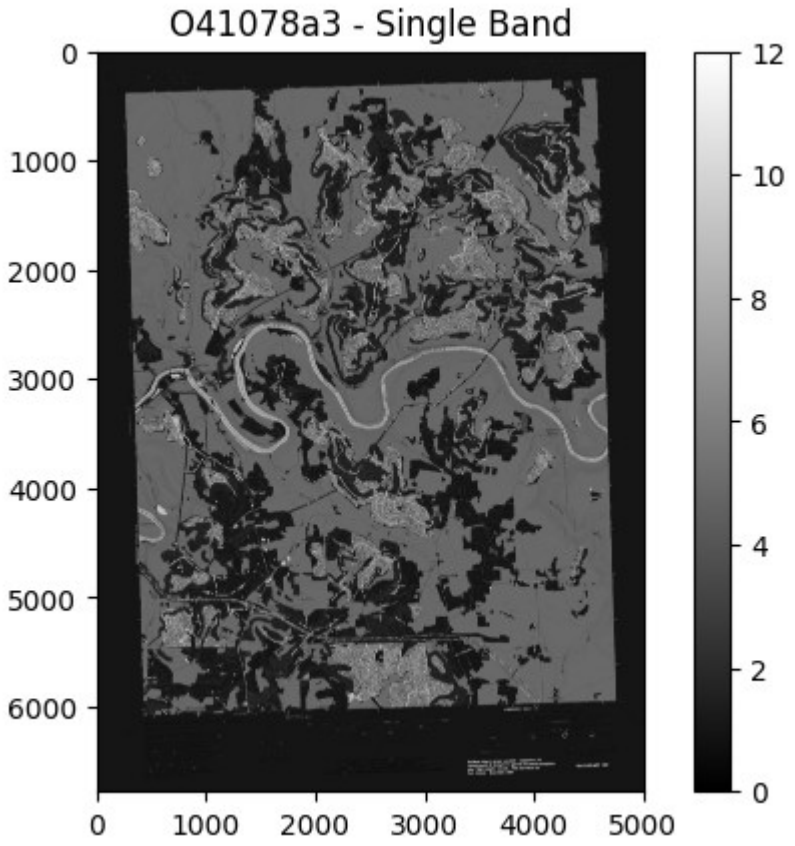
- NoData Handling in `rasterio`:
  - `rasterio` automatically manages NoData during read/write
  - Example: Setting NoData values and masking them in analysis



Practical Example – Working with Raster Data in `rasterio`

- Example: Working wirh Raster Data (PGD Chapter 3).ipynb

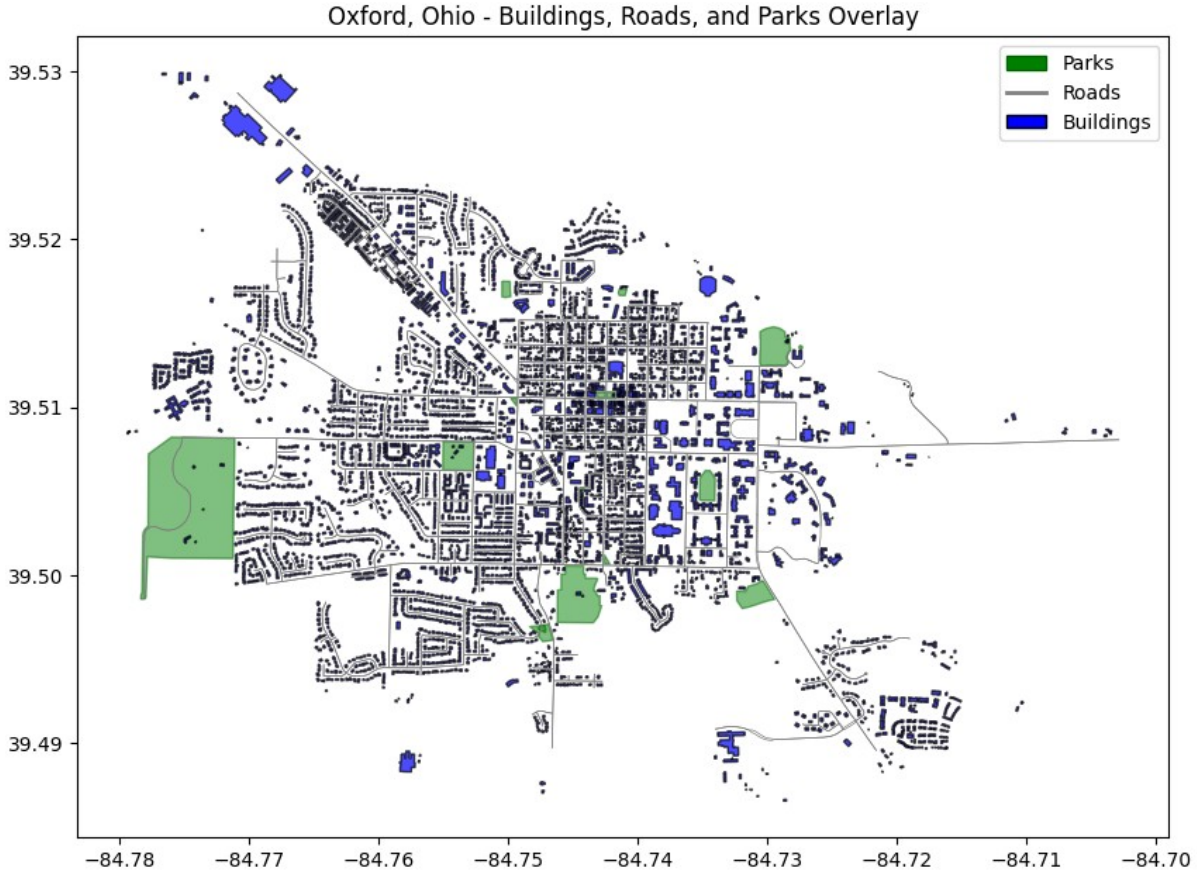





```
Metadata for data/o41078a1.tif:
  - Number of Bands: 1
  - Width: 5032 pixels
  - Height: 6793 pixels
  - CRS: EPSG:32617
  - Bounds: BoundingBox(left=740726.43745
  - Driver: GTiff
  - Data Type: uint8
  - Transform:
| 2.44, 0.00, 740726.44|
| 0.00,-2.44, 4557390.41|
| 0.00, 0.00, 1.00|
```

Practical Example: Working with Vector Data

- Working with Vector Data: Pulling from OpenStreetMap.ipynb



Geospatial Projections

- Geospatial data maps Earth's 3D surface to a 2D plane.
- Projections transform latitude/longitude into x, y coordinates.
- Different projections are optimized for specific tasks (e.g., navigation, area preservation).
- Main challenge is converting data between different projections and datums.

Working with Projections in Python

- The `pyproj` library handles projection transformations.
- Built on PROJ.4, a widely used geospatial library.
- Core classes:
  - Proj (for coordinate transformations) and
  - Geod (for great-circle distance calculations).
- Supports conversion between different projections with ease.

```
import pyproj
print(pyproj.__version__)
```

Example

```
import pyproj

# Define projections
src_proj = pyproj.Proj(proj="longlat", ellps="WGS84", datum="WGS84")  # Latitude/Longitude
dst_proj = pyproj.Proj(proj='utm', zone=33, ellps='WGS84')  # UTM Zone 33N

# Example coordinates (Latitude, Longitude)
lat, lon = 41.9028, 12.4964  # Coordinates of Rome

# Convert from Lat/Lon to UTM
x, y = pyproj.transform(src_proj, dst_proj, lon, lat)

print(f"Projected Coordinates (UTM): {x}, {y}")
```

EPSG Codes

- **EPSG Codes**: Standardized reference codes for coordinate systems.
- Managed by the **International Association of Oil & Gas Producers (OGP)**.
- Each code corresponds to a specific geographic or projected coordinate system (e.g., WGS84 = EPSG:4326).
- Simplifies projection identification and transformations.

```
# Define source and target projections using EPSG codes
src_proj = pyproj.CRS.from_epsg(4326)  # EPSG:4326 is WGS84 (lat/lon)
dst_proj = pyproj.CRS.from_epsg(32633)  # EPSG:32633 is UTM Zone 33N (projected)
```

Converting Between Projections

- `pyproj` allows direct transformations between projections.
- Use the `Transformer` class to easily convert between coordinate systems.
- Handles both **individual** and **batch** coordinate transformations.

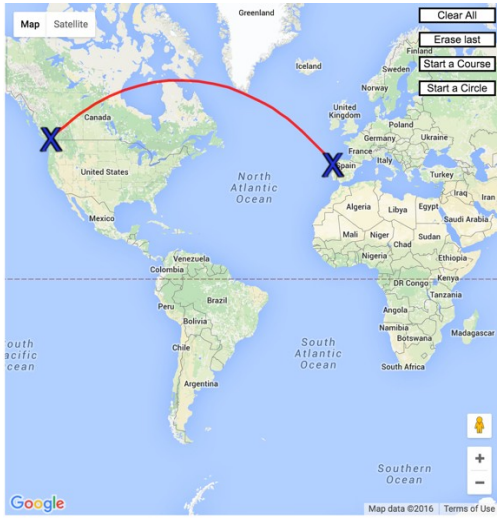
```
# Create a Transformer object
transformer = pyproj.Transformer.from_crs(src_proj, dst_proj)

# Numpy arrays of latitudes and longitudes (batch data)
lats = np.array([41.9028, 41.8902, 41.8986])  # Example latitudes
lons = np.array([12.4964, 12.4922, 12.4769])  # Example longitudes

# Transform all coordinates in one batch
x_coords, y_coords = transformer.transform(lats, lons)
```

The Proj Class

- **Purpose**: Transforms geographic coordinates (lat/lon) into projected coordinates (x, y) and vice versa.
- Customizable with different projections (e.g., Mercator, Transverse Mercator).
- **Example**: Create a Proj object with the WGS84 ellipsoid and transverse Mercator projection.

```
# Transverse Mercator with WGS84 ellipsoid
proj = pyproj.Proj(proj='tmerc', ellps='WGS84')


# Convert Lat/Lon to (x, y)
x, y = proj(12.4964, 41.9028)  # Rome coordinates (lon, lat)
```

The Geod Class



- **Purpose**: Handles geodetic computations, such as calculating distances and angles between points on the Earth's surface.
- Built for great-circle distance calculations based on the ellipsoid model.
- Key methods:
  - `fwd()`: Compute the endpoint from a starting point, direction (azimuth), and distance.
  - `inv()`: Calculate azimuth and distance between two points.

```python
geod = pyproj.Geod(ellps='WGS84')

# Start at a point (longitude, latitude)
lon1, lat1 = 12.4964, 41.9028  # Rome

# Compute point 10 km northeast (315 degrees)
angle = 315
distance = 10000  # meters
lon2, lat2, back_azimuth = geod.fwd(lon1, lat1, angle, distance)
```

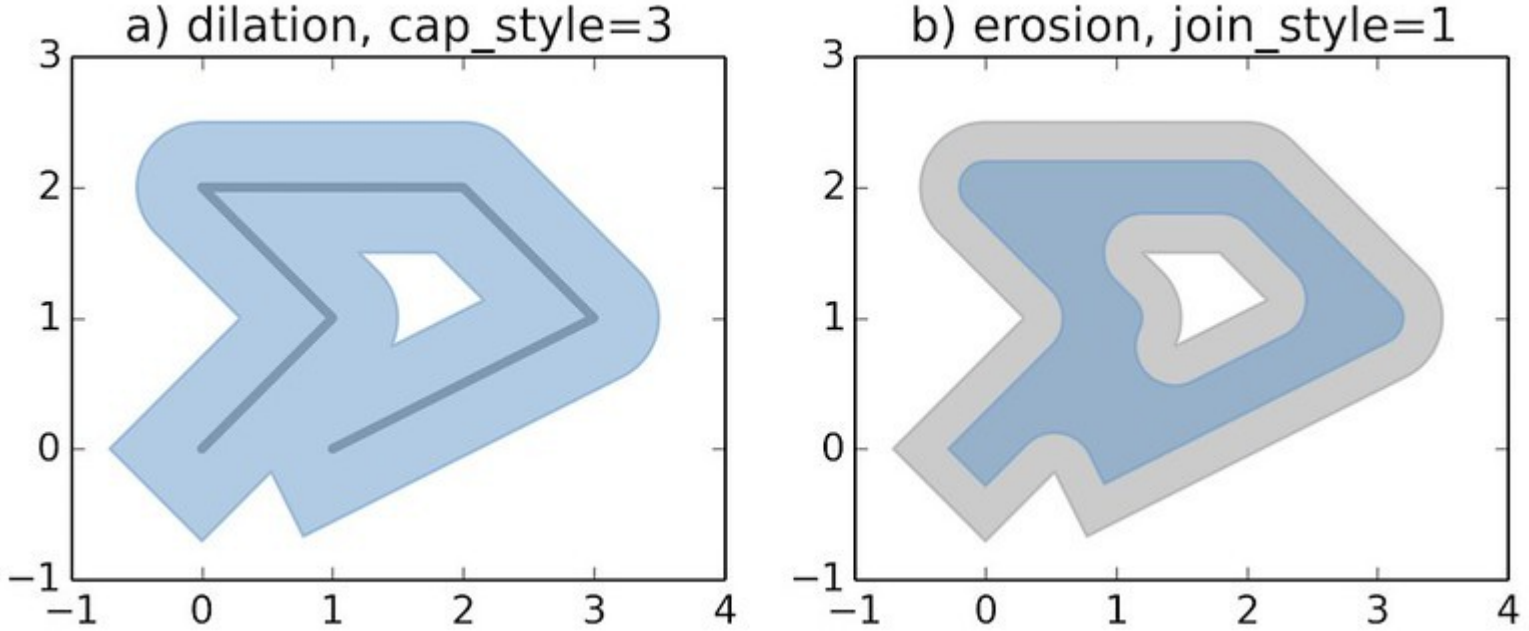
Where will I end up if I go straight at this angle?

```python
# Create a Geod object with WGS84 ellipsoid
geod = pyproj.Geod(ellps='WGS84')

# Coordinates of two points (longitude, latitude)
lon1, lat1 = 12.4964, 41.9028  # Rome
lon2, lat2 = 2.3522, 48.8566   # Paris

# Compute the forward and backward azimuths, and distance between points
fwd_azimuth, back_azimuth, distance = geod.inv(lon1, lat1, lon2, lat2)
```

What is the shortest distance between two points?

Analyzing Geospatial Data with Shapely

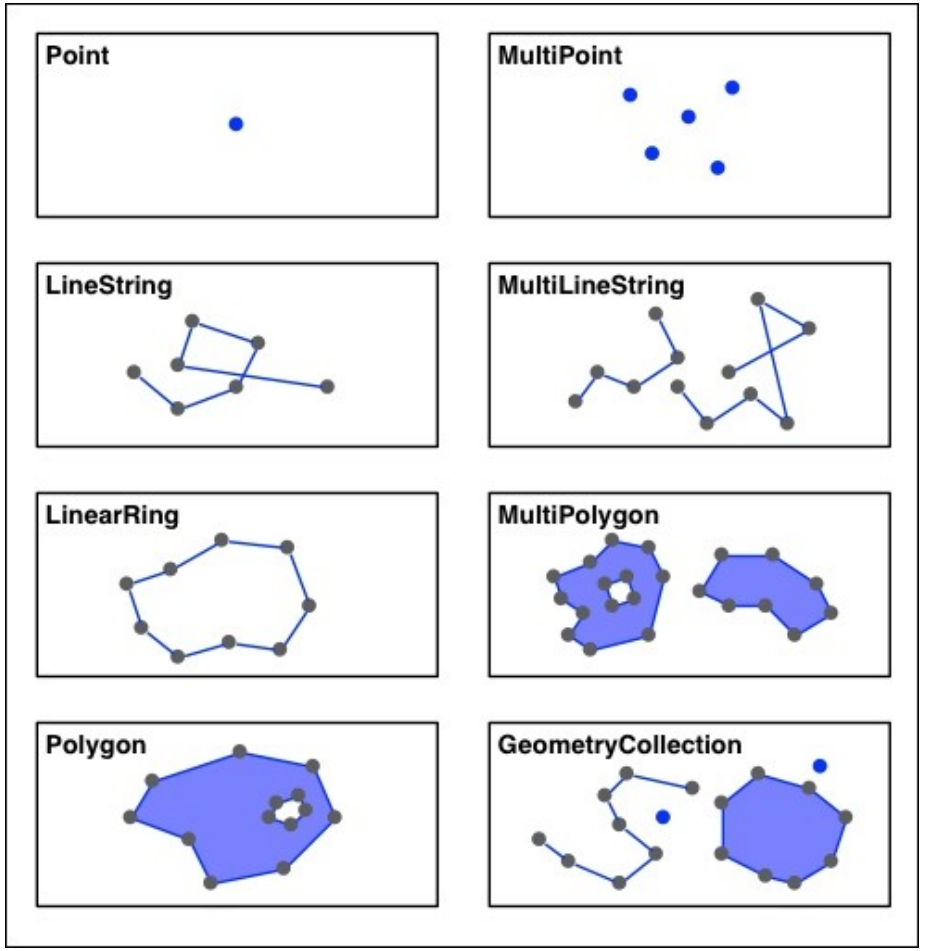
- **Shapely**: A Python library for 2D geometric operations.
- Based on the **GEOS** (Geometry Engine Open Source) library, which handles computational geometry in C++.
- Supports **points**, **lines**, **polygons**, and collections of these geometries.
- **Spatial vs. Geospatial**: Shapely works on a 2D Cartesian plane, not geographic coordinates.




```
import shapely
import shapely.speedups
print(shapely.__version__)
print(shapely.speedups.available)
```

Shapely Geometries

- Geometry Types:
  - **Point**: Single point in 2D or 3D space.
  - **LineString**: Sequence of points forming a line.
  - **Polygon**: Filled area, possibly with holes.
  - **MultiPoint, MultiLineString, MultiPolygon**: Collections of geometries.
  - **GeometryCollection**: Mixed collection of points, lines, and polygons.

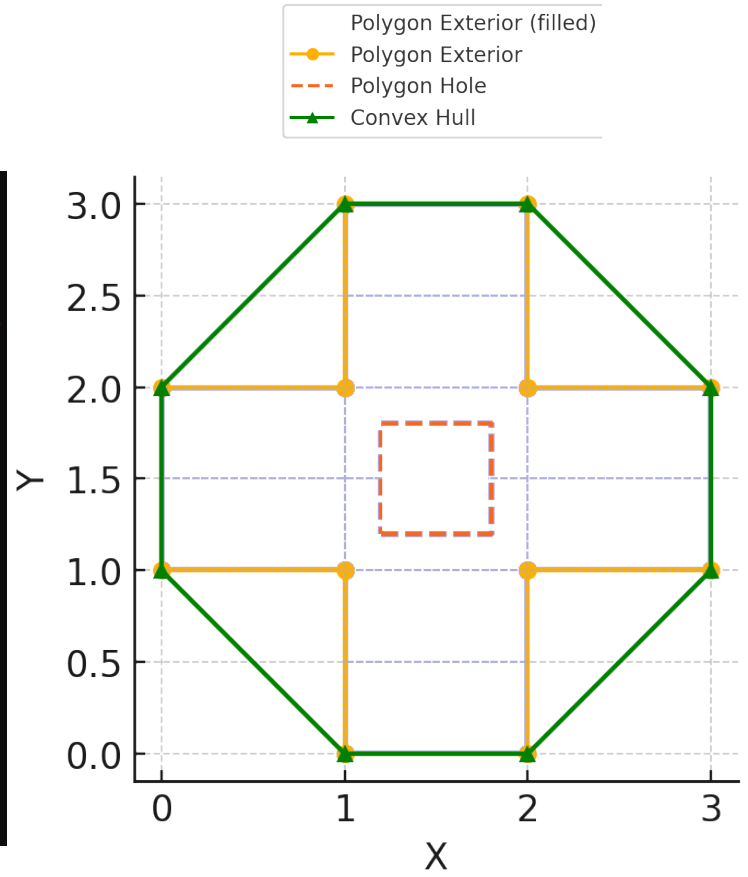



```python
import shapely.geometry

# Create geometries
point = shapely.geometry.Point(1, 1)
line = shapely.geometry.LineString([(0, 0), (1, 1), (2, 2)])
polygon = shapely.geometry.Polygon([(0, 0), (1, 0), (1, 1), (0, 1), (0, 0)])
```

Accessing Geometric Properties

- **Coordinates**: Access both exterior and interior (holes) of polygons.
- **Exterior & Interior**: Outer and inner boundaries of polygons.
- **Exterior & Interior**: Outer and inner boundaries of polygons.
- **Convex Hull**: Smallest convex shape that encloses a geometry.

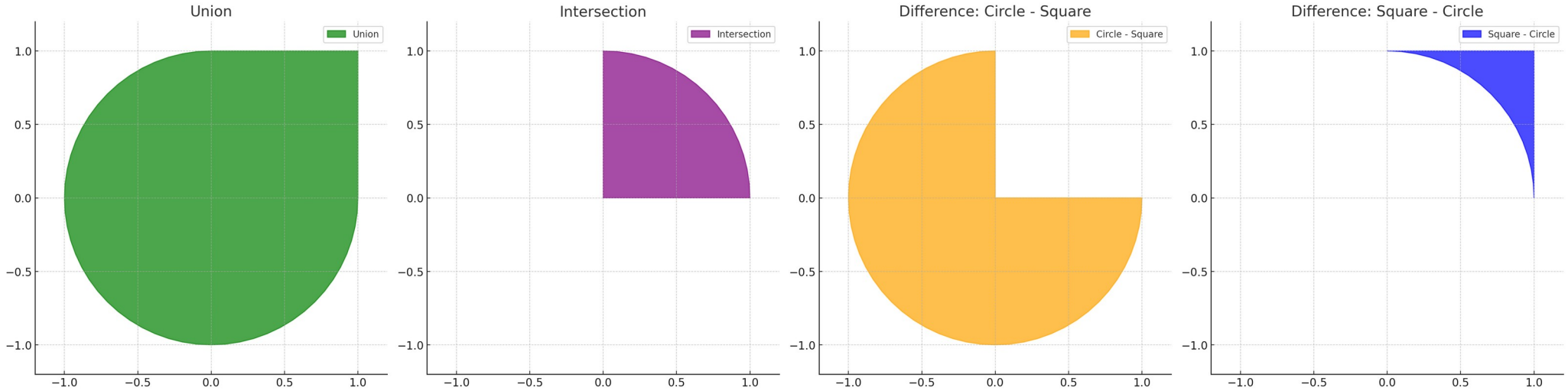
```
# Create a "plus"-shaped polygon with a hole
exterior = [(1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (2, 2), (2, 3),
            (1, 3), (1, 2), (0, 2), (0, 1), (1, 1), (1, 0)]  # Plus-shaped outer boundary
interior = [(1.2, 1.2), (1.8, 1.2), (1.8, 1.8), (1.2, 1.8), (1.2, 1.2)]  # Square hole

polygon_with_hole = shapely.geometry.Polygon(shell=exterior, holes=[interior])

# Extract coordinates for exterior, interiors, and convex hull
coords_exterior = list(polygon_with_hole.exterior.coords)
coords_interiors = [list(interior.coords) for interior in polygon_with_hole.interiors]
hull = polygon_with_hole.convex_hull
```

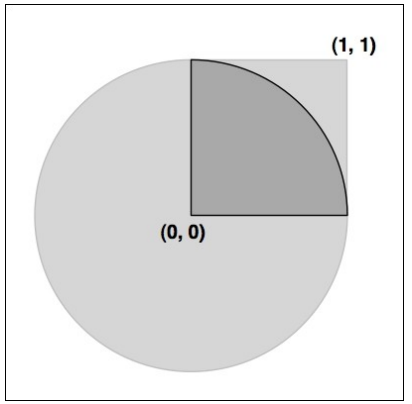


Shapely Geometric Operations

- **Intersection**: Finds the common area between geometries.
- **Union**: Combines multiple geometries into one.
- **Difference**: Subtracts one geometry from another.
- **Buffer**: Creates a surrounding area (buffer) around a geometr

```
pt = shapely.geometry.Point(0, 0)
circle = pt.buffer(1.0)

square = shapely.geometry.Polygon([(0, 0), (1, 0),
                                   (1, 1), (0, 1),
                                   (0, 0)])

intersect = circle.intersection(square)
```





Shapely for Real-World Geospatial Data

- **Shapely** can be integrated with libraries like **Fiona** and **GeoPandas** for working with real-world geospatial data.

Example Workflow:

1. Use **Fiona** or **GeoPandas** to load geospatial data.
2. Manipulate geometries with **Shapely** (e.g., buffer, intersection).
3. Save results back into a geospatial format.

```python
import geopandas as gpd
from shapely.geometry import Point

# Load a shapefile
gdf = gpd.read_file('data.shp')

# Create a Shapely Point and buffer it
point = Point(10, 10)
buffered_point = point.buffer(5)

# Find the intersection with geospatial data
intersection = gdf[gdf.intersects(buffered_point)]

# Save the result
intersection.to_file('output.shp')
```

(Keep only the features within a circle)

Common Uses of Shapely

- **Buffering**: Create buffer zones around points, lines, or polygons.
- **Distance Calculations**: Measure distances between geometries.
- **Simplification**: Reduce the complexity of a geometry (e.g., reduce vertices in polygons).
- **Intersection/Union**: Combine or intersect geometries to find overlapping areas.
- **Convex Hull**: Create the smallest convex polygon that contains all points of a geometry.
- **Point in Polygon**: Determine if a point lies within a polygon.
- **Centroid Calculation**: Find the geometric center of a polygon.
- **Closest Points**: Find the closest points between two geometries.

Tangent.... Transforming Geometries

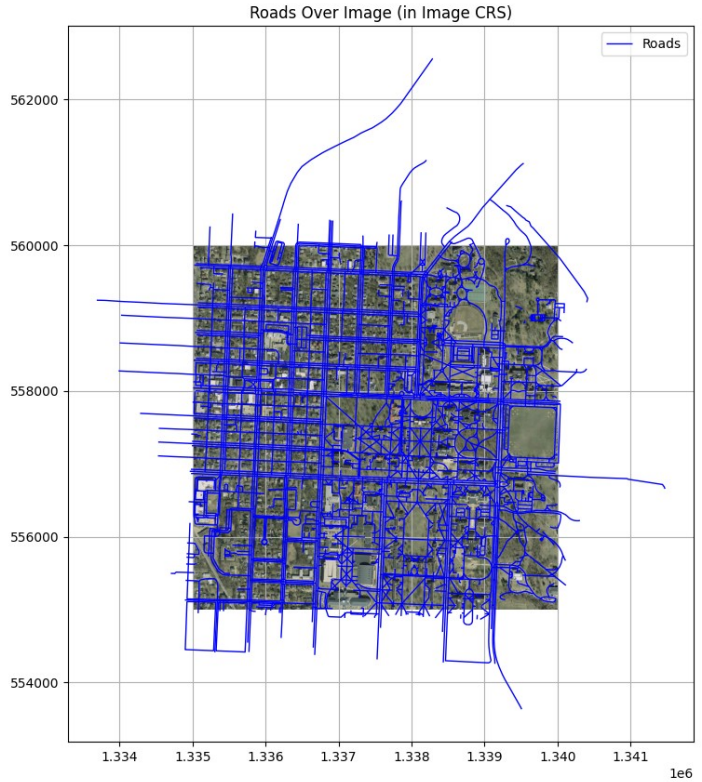
- **GeoPandas** handles CRS transformations natively without needing **Shapely** for this purpose.
- You can directly use GeoPandas to convert geometries from one CRS to another using the `to_crs()` method.
- **Shapely** is mainly used within GeoPandas for geometric operations like intersection, buffering, and simplifying geometries.

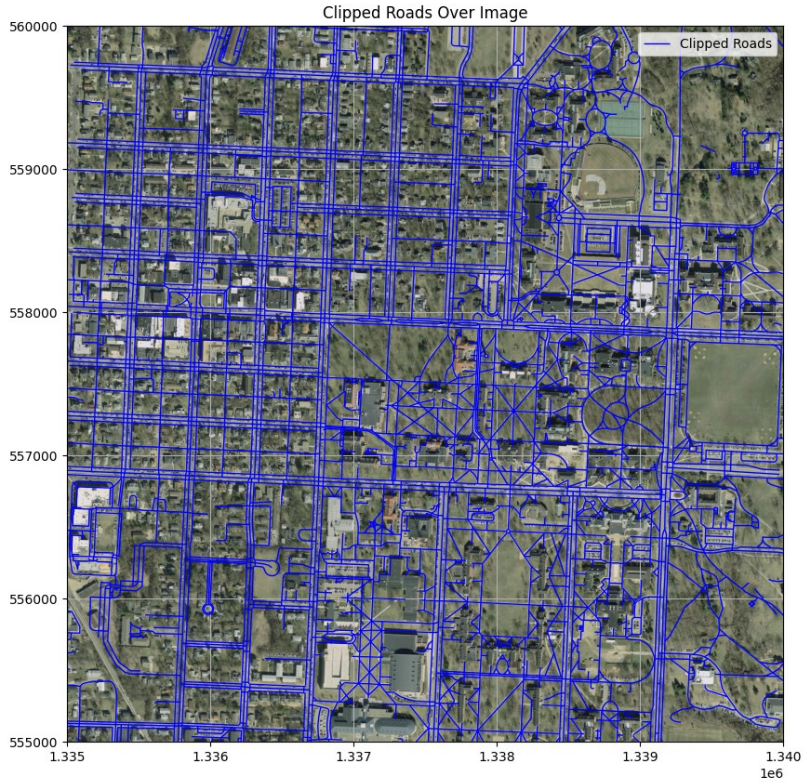
```python
import geopandas as gpd

# Load a GeoDataFrame with geometries in WGS84 (EPSG:4326)
gdf = gpd.read_file('data.shp')

# Transform geometries to UTM Zone 18N (EPSG:32618)
gdf_utm = gdf.to_crs(epsg=32618)

# Save the transformed geometries back to a file
gdf_utm.to_file('output_utm.shp')

print(gdf_utm.head())
```

Cropping GeoData to an Image Extent

Example Workflow:

1. Use **rasterio** to get the image bounds.
2. Convert the bounds to a polygon using **Shapely**.
3. Use **GeoPandas** to clip the data to the bounding box.





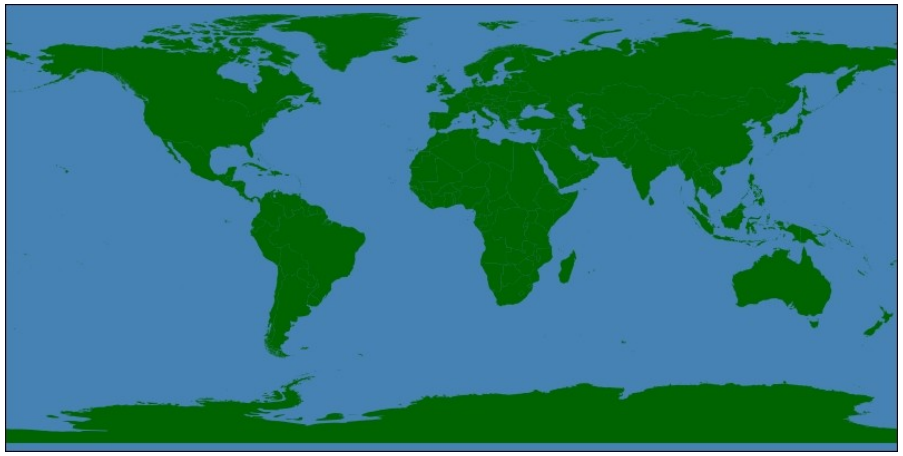
[Demo on Colab]

```python
# Create a Shapely bounding box from the image bounds
from shapely.geometry import box
raster_bounds_polygon = box(image_bounds.left,
                            image_bounds.bottom,
                            image_bounds.right,
                            image_bounds.top)
```

```python
# Clip the roads using the intersection method
roads_clipped = roads.intersection(raster_bounds_polygon)

# Filter out empty geometries (if any)
roads_clipped = roads_clipped[~roads_clipped.is_empty]
```

Visualizing Geospatial Data

- **Mapnik**: A powerful library for rendering geospatial data into images.
- **Converts data** from formats like **Shapefiles** into visually appealing maps.
- Features control with **rules** and **symbolizers:**
  - **Rules**: Define which features to display.
  - **Symbolizers**: Control how features are visually rendered (e.g., color, line thickness, etc.).
- **Mapnik** is widely used for high-quality map rendering (e.g., **OpenStreetMap**).
- **Matplotlib** and **Cartopy**: For simpler applications, **Matplotlib** combined with **Cartopy** can suffice for geospatial visualizations.



Review of Pandas & GeoPandas

- **Pandas** is a powerful Python library for data manipulation and analysis.
- Think of it as **"Excel in Python"**:
  - Similar to working with tables in Excel but with much more flexibility and power.
- Built around two primary data structures:
  - **Series**: One-dimensional labeled array (like a column in a table).
  - **DataFrame**: Two-dimensional, tabular data structure with labeled axes (rows and columns).

Key Features:

- Easy handling of **missing data**.
- Powerful **grouping** and **aggregation** functions.
- Intuitive handling of **time series** data.
- Integration with other libraries like **Matplotlib** for visualization.

```python
import pandas as pd

# Create a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Paris', 'London']}
df = pd.DataFrame(data)

# Display the DataFrame
print(df)
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Paris
2	Charlie	35	London

Understanding Pandas Index

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Paris
2	Charlie	35	London

- **Index**: A unique identifier for each row (like row numbers in Excel).
  - Can be based on default numeric values or custom labels.
  - Acts as a reference for rows during operations like filtering, grouping, and merging.

```
1  df.sample(3)
```

	Name	Age	City
2	Charlie	35	London
1	Bob	30	Paris
0	Alice	25	New York

Key Functions:

- **Reset Index**:

```
df.reset_index()  # Resets the index to default numeric values
```

- **Set a Custom Column as the Index**:

```
df.set_index('Name')  # Use the 'Name' column as the index
```

```
1  df.set_index('Name')
```

Name	Age	City
Alice	25	New York
Bob	30	Paris
Charlie	35	London

Accessing Data in Pandas

- **Accessing Columns**:
  - Use square brackets to access columns by name (like selecting a column in Excel).

```
df['Name']  # Access the 'Name' column
```

- **Accessing Rows**:
  - Use `iloc[]` for row index-based access and `loc[]` for label-based access.

```
df.iloc[0]  # Access the first row
df.loc[2]   # Access the row with index label 2
```

- **Accessing a Specific Item**:
  - Combine row and column access to fetch a specific item.

```
df.iloc[0, 1]  # Access the item in the first row and second column
```

.

Filtering Data in Pandas

- Use **Boolean conditions** to filter data based on column values (like Excel filters).

```
df_filtered = df[df['Age'] > 30]  # Filter rows where 'Age' is greater than 30
```

- Chaining conditions with & (AND) or | (OR):

```
df_filtered = df[(df['Age'] > 30) & (df['City'] == 'Paris')]
```

- Note – the expression `df['Age']>30` is a series of Booleans

Grouping and Aggregation

- Group data by one or more columns (like Excel's **PivotTable**).

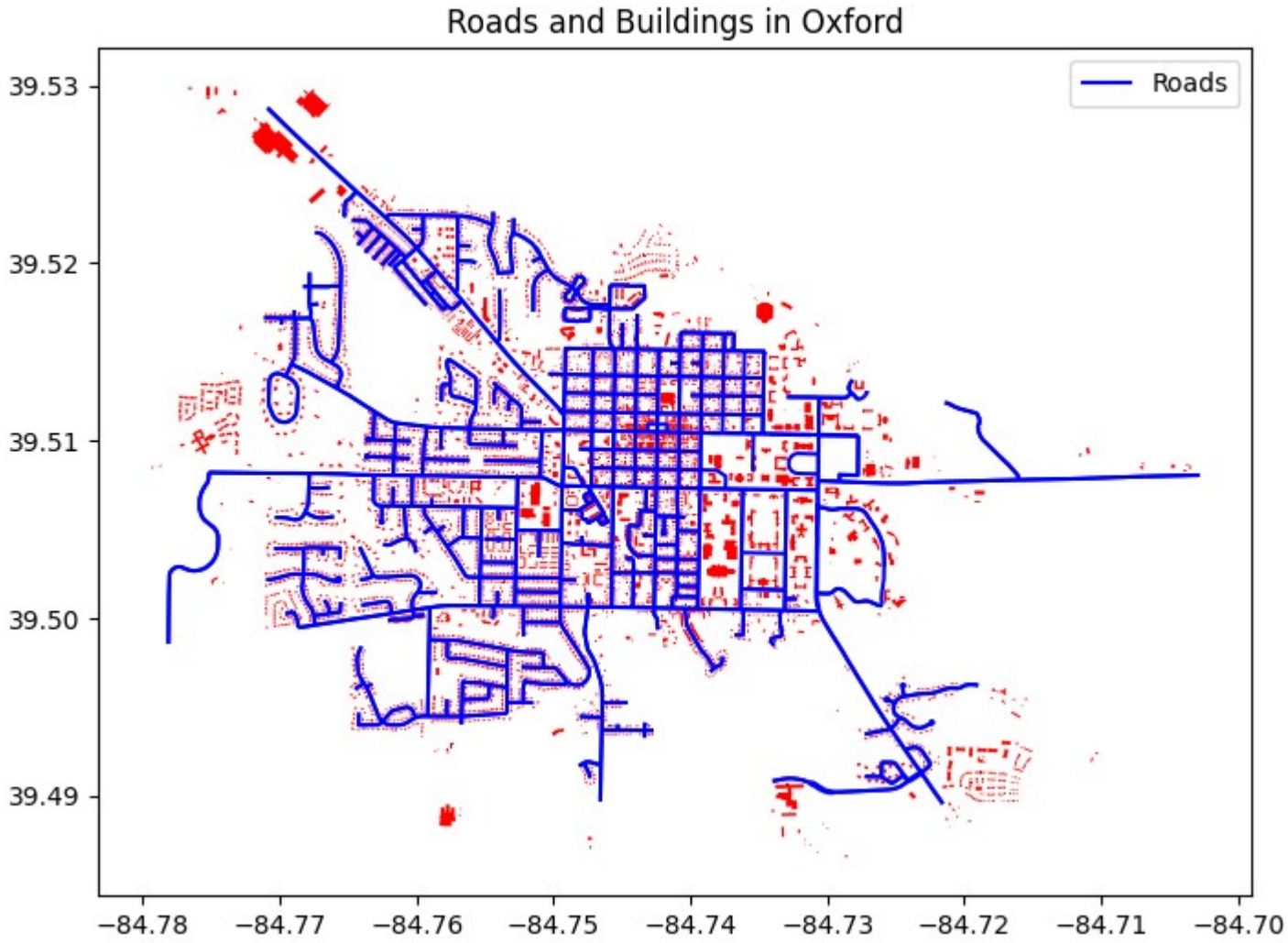
```
df_grouped = df.groupby('City')['Age'].mean()  # Get average 'Age' per 'City'
```

- Other aggregations: sum(), count(), min(), max(), etc.

```
df_grouped = df.groupby('City')['Age'].sum()  # Total 'Age' per 'City'
```

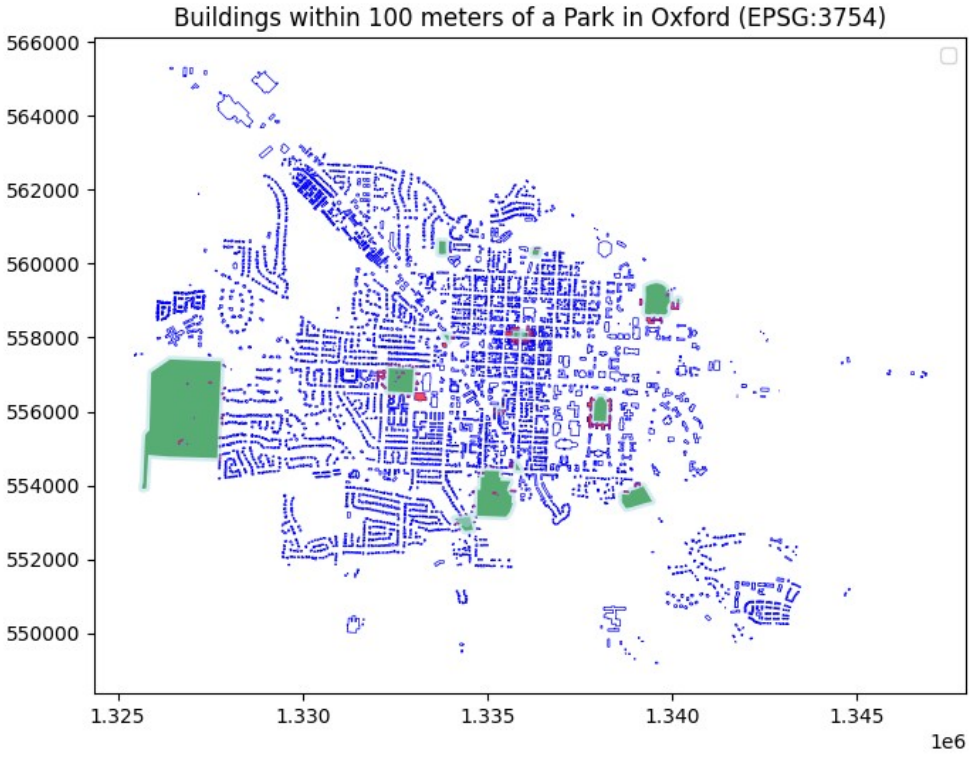
Introduction to GeoPandas

- **GeoPandas**: Extends **Pandas** to handle **geospatial data**.
- GeoPandas simplifies the process of loading, manipulating, and visualizing geospatial data.
- Can quickly visualize geographic features (e.g., roads, buildings) on a map.



Handling Spatial Relationships in GeoPandas

- **Spatial Relationships**: GeoPandas allows you to analyze spatial data relationships between geometries (e.g., intersecting, within, touching).



GeoSeries – The Building Block of GeoPandas

- A **GeoSeries** is like a Pandas Series but specifically for **geometries**
  - Can contain Points, LineStrings, or Polygons.
  - Each element is a **shapely.geometry** object.
- A **GeoDataFrame** is an extended Pandas DataFrame with a geometry column

```
import geopandas as gpd
from shapely.geometry import Point


# Create a GeoSeries of Points
geoseries = gpd.GeoSeries([Point(1, 2), Point(2, 3), Point(3, 4)])
print(geoseries)
```

Spatial Operations in GeoPandas

- GeoPandas enables **spatial operations** like:
  - **intersects()**: Check if geometries overlap.
  - **contains()**: Check if one geometry contains another.
  - **buffer()**: Create a buffer zone around geometries.

```
# Create a buffer around a point
buffered = gdf.buffer(100)  # 100-meter buffer
print(buffered)
```

Coordinate Reference Systems (CRS)

- GeoPandas supports **CRS** transformations
  - CRS define how coordinates map to locations on the Earth.
  - Easily convert between CRSs using **to_crs()**.

```
# Reproject GeoDataFrame to EPSG:4326 (WGS84)
gdf = gdf.set_crs(epsg=3754)  # Current CRS
gdf_reprojected = gdf.to_crs(epsg=4326)  # Convert to WGS84
```

Working with Geospatial Files (Shapefiles, GeoJSON)

- GeoPandas provides easy loading and saving of geospatial formats:
  - **read_file()**: Load data from Shapefiles, GeoJSON, etc.
  - **to_file()**: Save data in geospatial formats.

```
# Load a GeoJSON file
gdf = gpd.read_file('oxford_buildings.geojson')


# Save it as a Shapefile
gdf.to_file('buildings.shp')
```

Visualization of Geospatial Data with GeoPandas

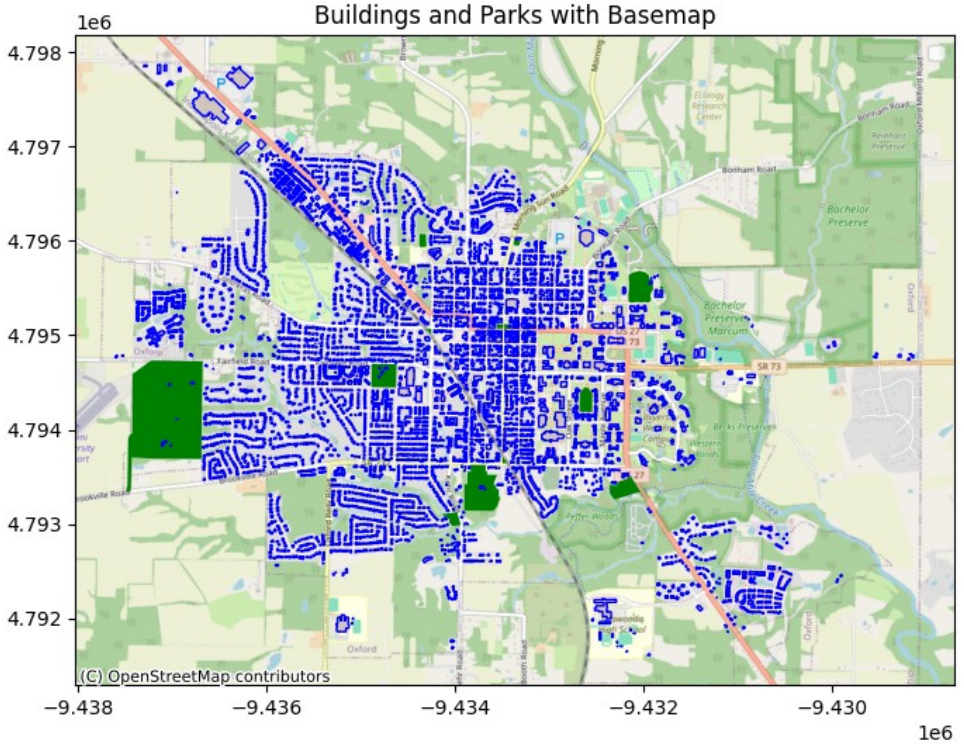
- GeoPandas provides easy visualization with **Matplotlib** integration
  - **plot()**: Built-in function to plot geospatial data.
  - Supports **coloring by attribute**, **layering**, and custom **legends**.

```
# Plot buildings and parks on the same plot
fig, ax = plt.subplots(figsize=(10, 6))
parks.plot(ax=ax, color='green', label='Parks')
buildings.plot(ax=ax, edgecolor='blue', facecolor='none', label='Buildings')


plt.legend()
plt.title('Buildings and Parks in Oxford')
plt.show()
```

Advanced Visualization with GeoPandas and Matplotlib

- GeoPandas allows advanced visualizations using Matplotlib features
- **Adding basemaps:** Use libraries like contextily to add tile maps for better visualization.

```
import contextily as ctx
fig, ax = plt.subplots(figsize=(10, 6))
parks.plot(ax=ax, color='green', label='Parks')
buildings.plot(ax=ax, edgecolor='blue', facecolor='none', label='Buildings')

# Add a basemap (e.g., OpenStreetMap)
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)

plt.legend()
plt.title('Buildings and Parks with Basemap')
plt.show()
```



Adding Columns

- You can programmatically **add columns** to a GeoDataFrame, such as calculated fields (e.g., area, distance).

```
# Load GeoJSON for parks
parks = gpd.read_file('oxford_parks.geojson')


# Add a new column 'area' (in square meters) calculated for each geometry
parks['area'] = parks.geometry.area
```

```
# Calculate the distance from each building to the nearest park
buildings['distance_to_park'] = buildings.geometry.apply(
    lambda building: parks.distance(building).min()
)
```

Adding New Records (Geometries) to a GeoDataFrame

- You can programmatically **add new shapes** (points, polygons, etc.) as rows (records) in a GeoDataFrame.
- New shapes (e.g., Points, Polygons) can be added as rows to an existing GeoDataFrame using append().

```
# Define a new polygon (a square)
new_polygon = Polygon([(0, 0), (1, 0), (1, 1), (0, 1)])


# Create a new GeoDataFrame with the new polygon
new_park = gpd.GeoDataFrame({
    'Name': ['New Park'],
    'geometry': [new_polygon]
}, geometry='geometry')


# Append the new polygon to the existing parks GeoDataFrame
parks = parks.append(new_park, ignore_index=True)
```

Rasterio

- **Rasterio** is a Python library designed for working with geospatial raster data.
- It allows you to **read**, **write**, and **manipulate** raster data (e.g., satellite images, digital elevation models).
- Support for popular raster formats (GeoTIFF, JPEG, PNG).
- Handles metadata and coordinate reference systems (CRS).
- Allows reading specific bands or creating masks.

```python
import rasterio


# Open a GeoTIFF file
raster = rasterio.open('s1335555.tif')


# Print raster metadata
print(raster.meta)
```

```
{'count': 4,
 'crs': CRS.from_epsg(3754),
 'driver': 'GTiff',
 'dtype': 'uint8',
 'height': 5000,
 'nodata': None,
 'transform': Affine(1.0, 0.0, 1335000.0,
       0.0, -1.0, 560000.0),
 'width': 5000}
```

Reading Raster Data with Rasterio

- Rasterio can read raster data by **bands** (e.g., RGB images) or read the entire dataset.

```
# Read the first band (greyscale or the red band in RGB)
band1 = dataset.read(1)


# Read all bands (for multi-band images)
all_bands = dataset.read()
```

Metadata and CRS in Rasterio

- Rasterio gives you access to a raster's **metadata** and **coordinate reference system (CRS)**.
- You can use this information to align raster data with other geospatial datasets.

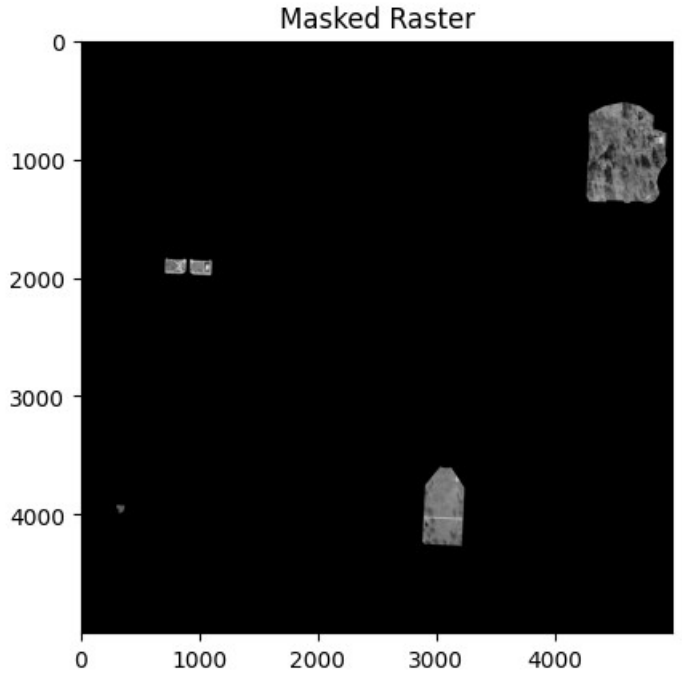
```
# Get metadata (CRS, dimensions, bands, etc.)
with rasterio.open('s1335555.tif') as dataset:
    print(f"CRS: {dataset.crs}")
    print(f"Width, Height: {dataset.width}, {dataset.height}")
    print(f"Number of Bands: {dataset.count}")
```

```
CRS: EPSG:3754
Width, Height: 5000, 5000
Number of Bands: 4
```

Masking and Cropping Rasters

- Rasterio allows you to **mask** or **crop** rasters based on regions of interest (ROI).
- This is useful for focusing on specific areas in large datasets.

```
# Convert parks to raster's CRS
parks = parks.to_crs(crs=raster.crs)

# Mask the raster using the park geometries
with rasterio.open('s1335555.tif') as src:
    out_image, out_transform = mask(src, parks.geometry, crop=True)

# Plot the masked raster
plt.imshow(out_image[0], cmap='gray')
```



Writing and Exporting Rasters

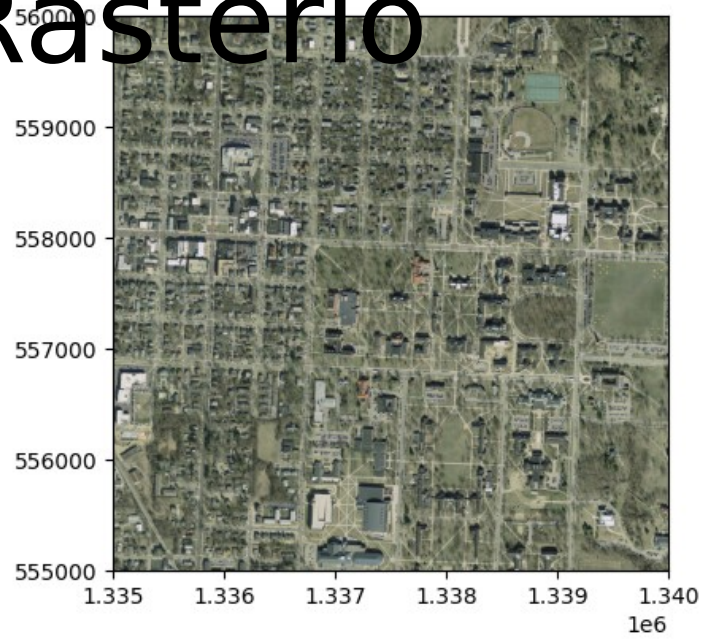
- Rasterio lets you **write** and **export** raster data to formats like GeoTIFF.
- You can also modify metadata, such as CRS and affine transformations.

```
import rasterio


# Create a new GeoTIFF from an array
with rasterio.open(
    'output.tif', 'w',
    driver='GTiff',
    height=out_image.shape[1],
    width=out_image.shape[2],
    count=1,
    dtype=out_image.dtype,
    crs=src.crs,
    transform=out_transform
) as dst:
    dst.write(out_image[0], 1)
```

Reprojecting Raster Data in Rasterio

- Rasterio supports **reprojecting** raster data to different CRSs.
- This is useful for aligning raster data with other geospatial datasets in different projections.

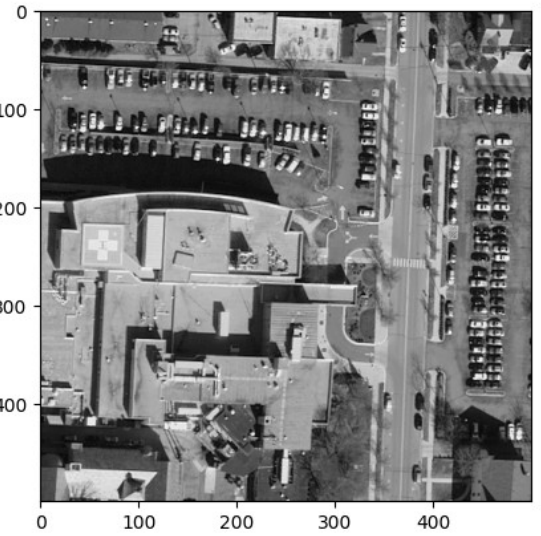
```
transform, width, height = calculate_default_transform(
    src.crs, 'EPSG:4326', src.width, src.height, *src.bounds)

reproject(
    source=rasterio.band(src, i),
    destination=rasterio.band(dst, i),
    src_transform=src.transform,
    src_crs=src.crs,
    dst_transform=transform,
    dst_crs='EPSG:4326',
    resampling=Resampling.nearest)
```



Windows vs Bounds in Rasterio

- **Windows**: Subset the raster using **pixel indices** (i, j).
- **Bounds**: Subset the raster using **geographical coordinates**.
- Windows are useful for high performance; bounds for working with map extents.

```python
from rasterio.windows import from_bounds

# Open raster and read data using geographical bounds
with rasterio.open('s1335555.tif') as dataset:
    bounds = (1336000.0, 558500.0, 1336500.0, 559000.0)  # Define geographic bounds
    window = from_bounds(*bounds, dataset.transform)
    bound_data = dataset.read(1, window=window)
```



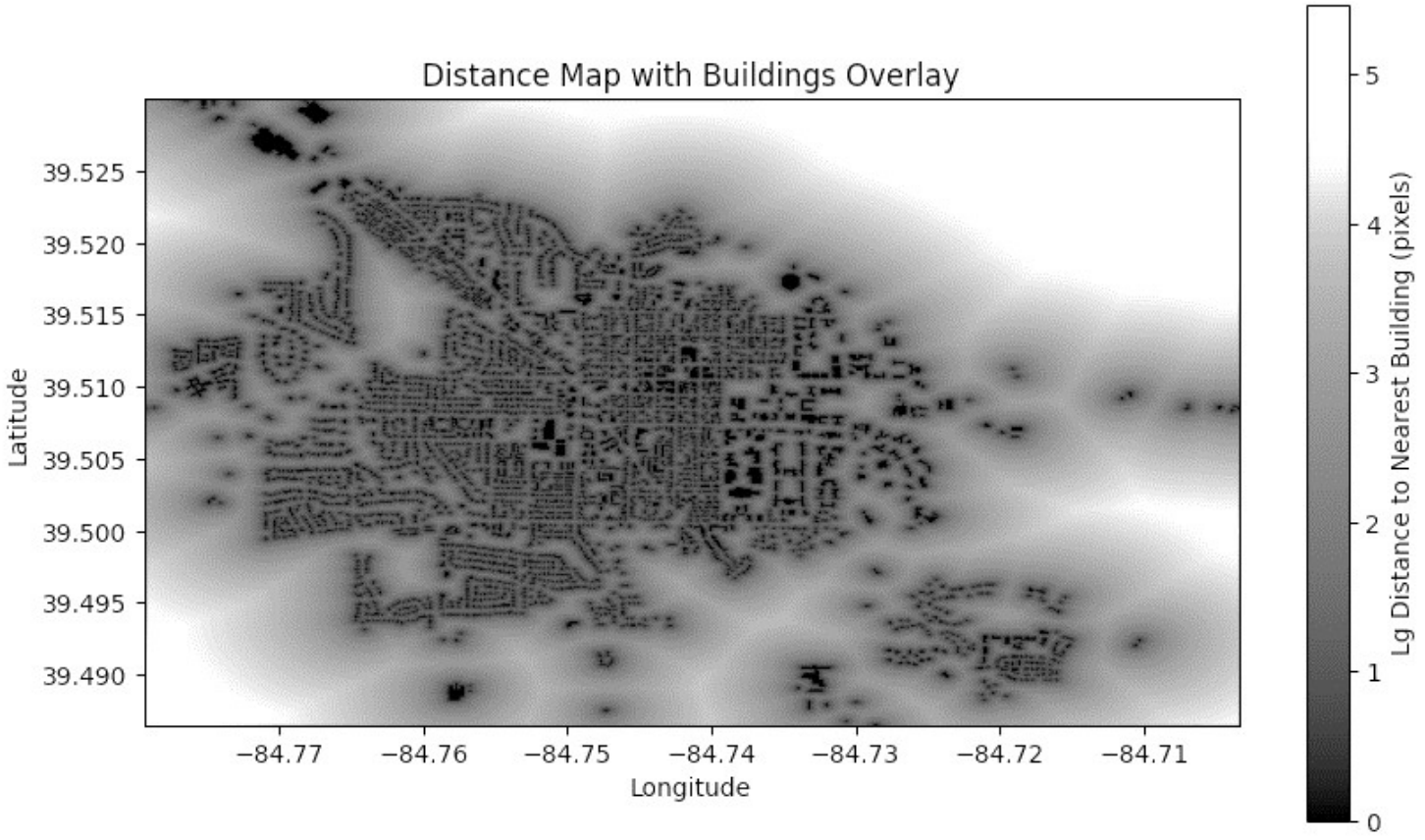
Rasterization in Rast

- **Rasterization** converts vector data (points, lines, polygons) into raster format.


```python
# Set up the raster size and bounds based on the buildings' extents
bounds = buildings.total_bounds  # Get the bounding box of the buildings (xmin, ymin, xmax, ymax)
out_shape = (500, 500)  # Define the raster size (height, width)
transform = from_bounds(*bounds, out_shape[1], out_shape[0])  # Create affine transform from bounds

# Rasterize the buildings (each building will have a value of 1 in the raster)
raster = rasterize(
    [(geom, 1) for geom in buildings.geometry],  # Assign value 1 to each building polygon
    out_shape=out_shape,  # Define the output shape
    fill=0,  # Fill the rest of the area with 0
    transform=transform  # Affine transformation for mapping to the raster grid
)
```

Computing Distance Maps with Rasterio

- **Distance maps** calculate the distance from each pixel to a reference point or feature (e.g., roads, buildings).




```
# Compute the distance map: distance from each pixel to the nearest building
distance_map = distance_transform_edt(raster == 0)  # Invert to compute distance from non-buildings
```

Vectorization in Rasterio

- **Vectorization** converts raster features (e.g., connected components) into vector geometries (points, polygons).
- The `rasterio.features.shapes()` function generates vector shapes (e.g., polygons) from raster regions with the same value.
- It returns a **generator** of tuples, where each tuple contains
  - A **geometry** (polygon in GeoJSON format).
  - The **value** of the corresponding region in the raster.

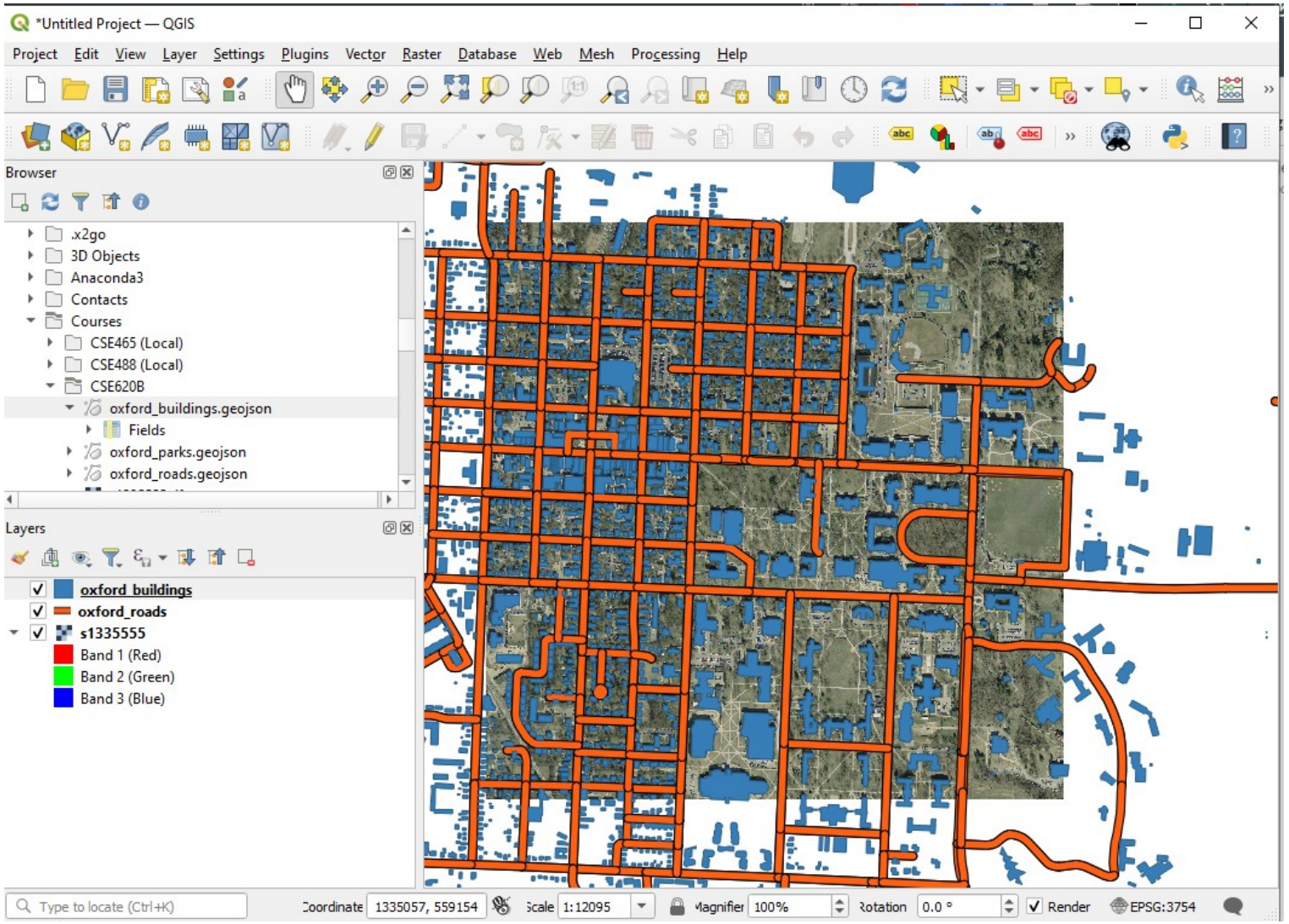
```
results = (
    {'properties': {'value': value}, 'geometry': shape(geom)}
    for geom, value in shapes(raster, transform=transform)
    if value == 1  # Only vectorize the building pixels (value 1)
)
```

What is QGIS?

- Free and open-source Geographic Information S
- Visualize and analyze spatial and remote sensing data.
- Supports vector and raster data formats.
- Developed by the QGIS Project, a global volunteer-driven community since 2002.
- Supports vector and raster data formats, with robust (python) plugin support for extensions.
- Free alternative to ESRI ArcGIS
- Widely used in academia for flexibility and cost-efficiency

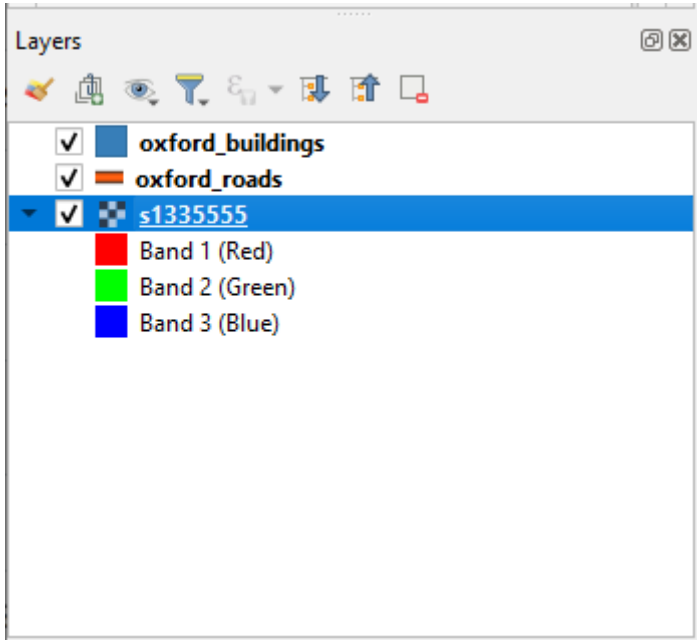
QGIS in Remote Sensing Projects

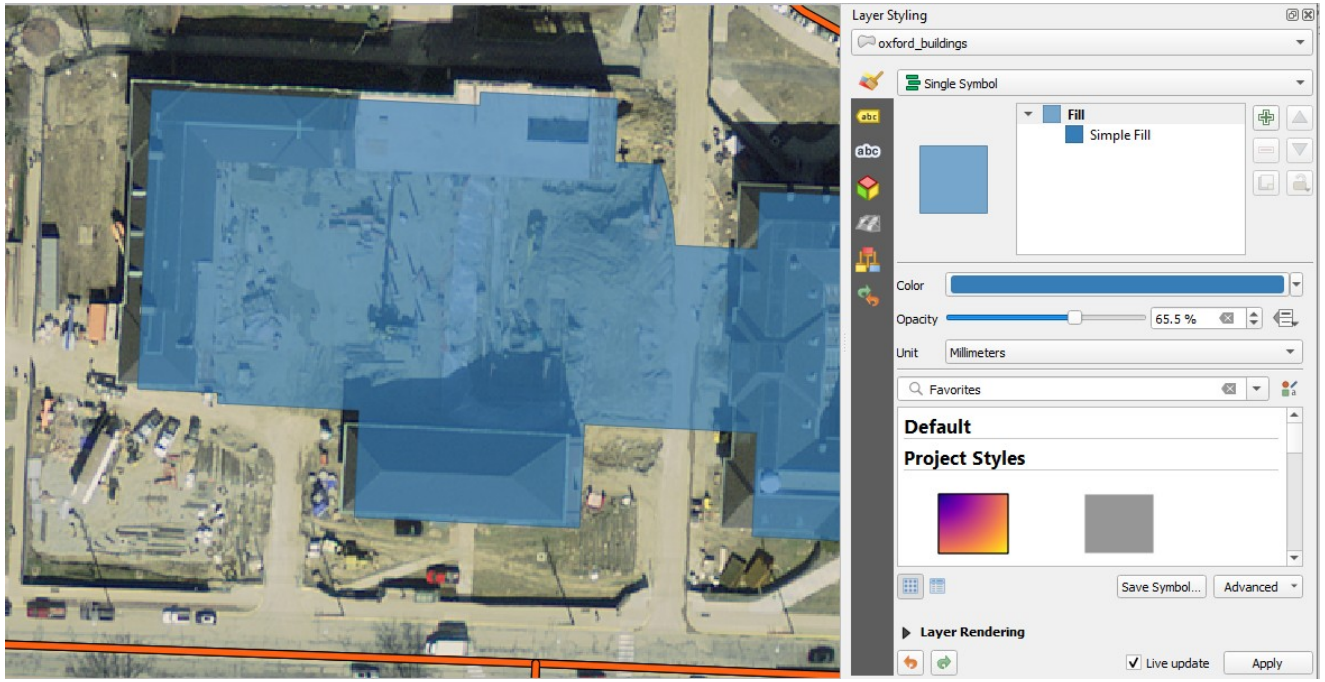
- Interactively visualize inputs like satellite imagery and sensor data.
- Analyze outputs such as land cover classifications or change detection.
- Combine multiple datasets for richer analysis (e.g., imagery, elevation, roads).



Working with Layers in QGIS

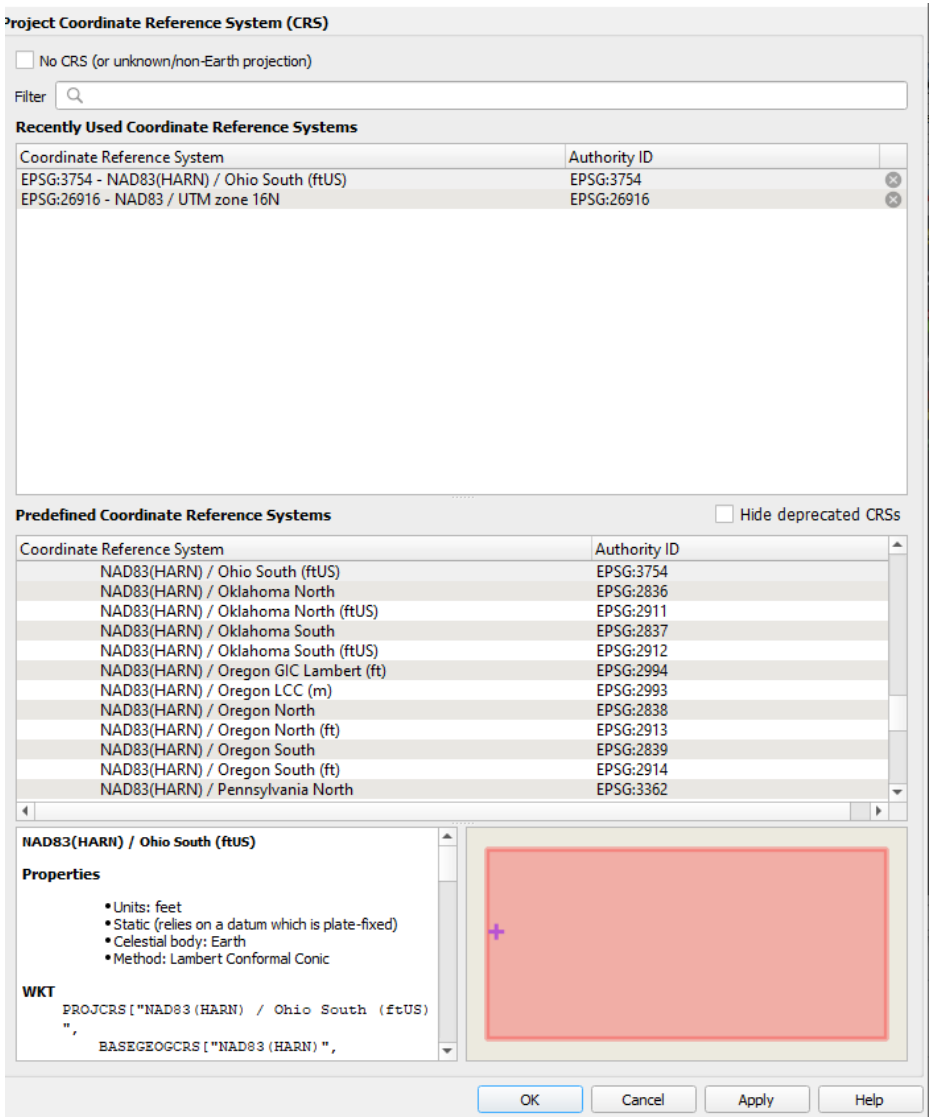
- Add multiple vector (e.g., shapefiles) and raster (e.g., GeoTIFF) layers.
- Adjust layer order by dragging them in the Layers Panel.
- Set transparency and visibility to emphasize important data.





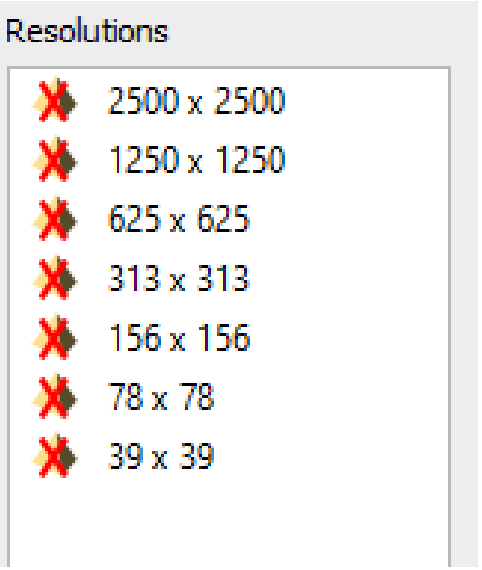
Coordinate Reference Systems (CRS) in QGIS

- Set the map's CRS to control how data is displayed.
- QGIS automatically reprojects layers to the map's CRS if they use different projections.
- Ensure consistent spatial alignment by setting the correct CRS (e.g., EPSG:4326 for WGS84).



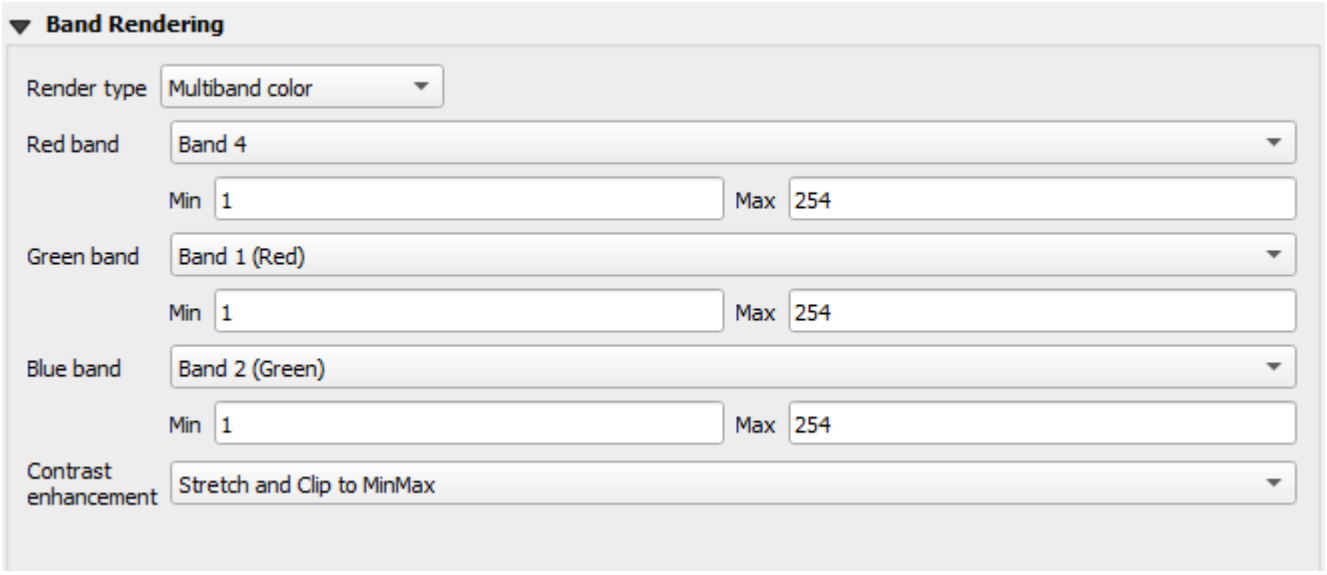
Improving Performance with Raster Pyramids

- Pyramids are reduced-resolution layers that speed up the display of large rasters.
- Useful for zooming and panning large datasets like high-resolution satellite imagery.
- Create pyramids through the raster layer properties.



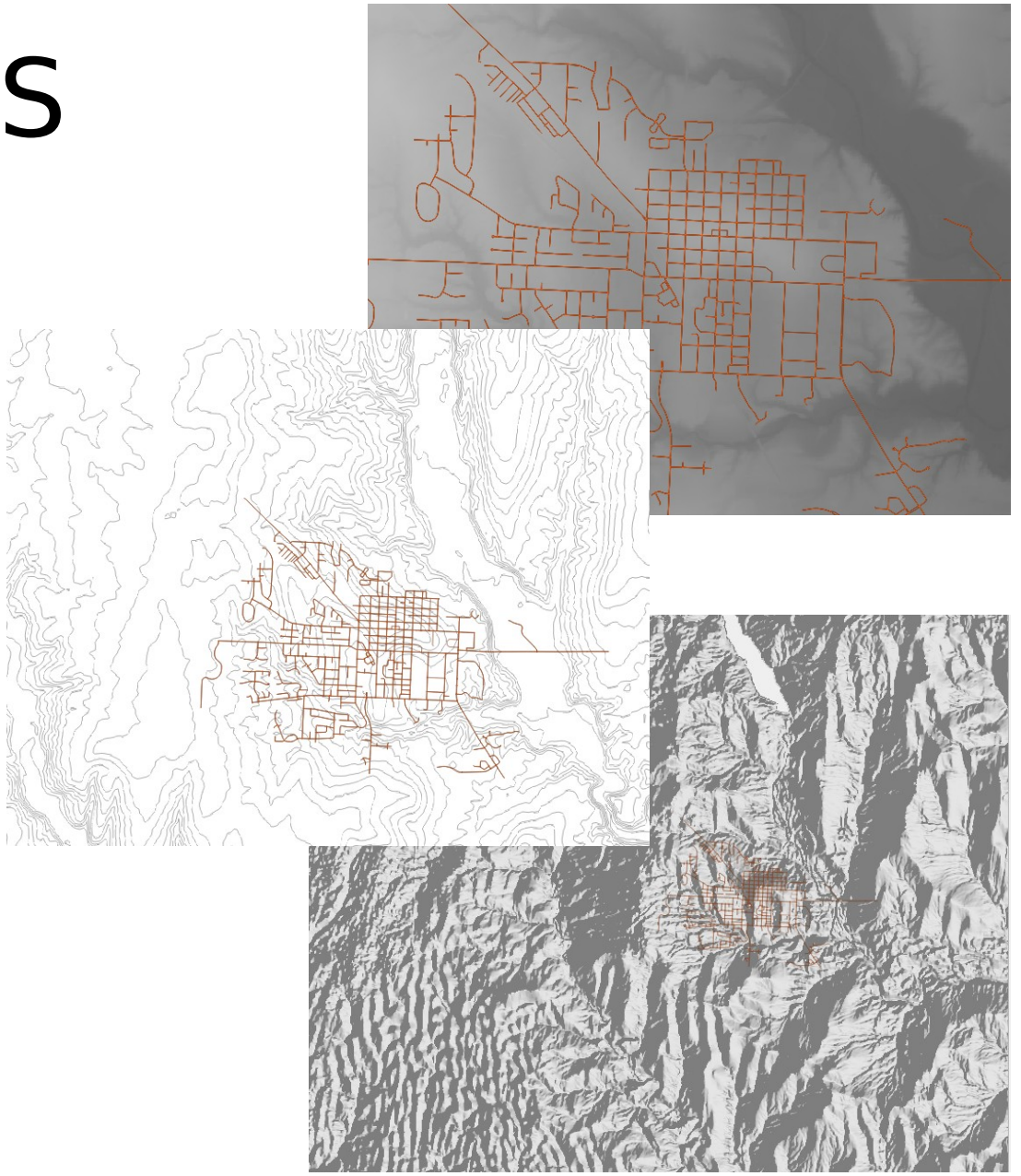
Color Mapping for Raster Data

- Remote sensing imagery typically has multiple bands (e.g., red, green, blue, infrared).
- Map bands to colors for different visualizations (e.g., true color, false color).
- Common for NAIP imagery: (Red, Green, Blue, NIR)
  - (NIR, Red, Green)
    - Red → Band 4
    - Green → Band 1
    - Blue → Band 2



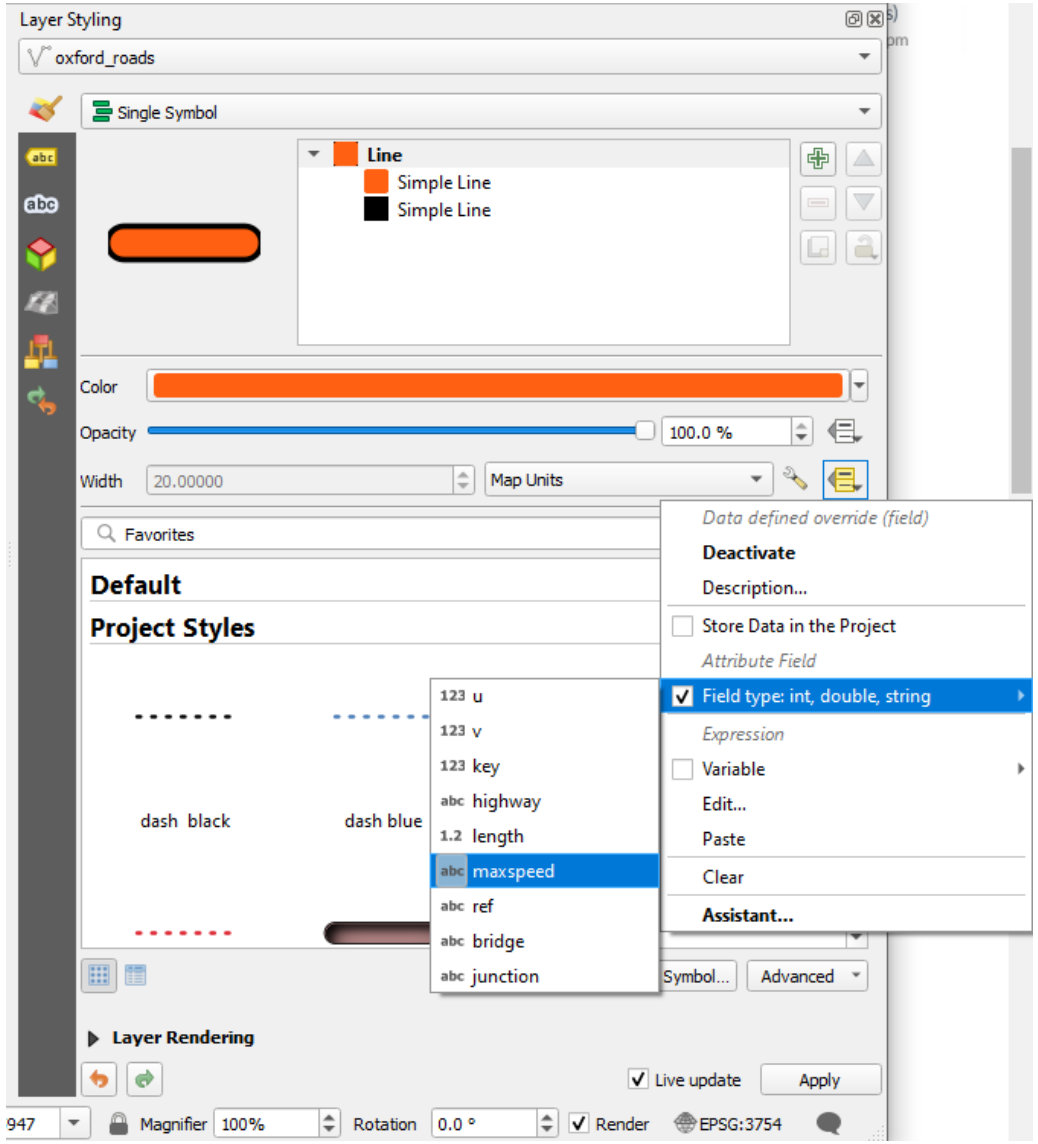
Hillshading in QGIS

- Hillshading simulates how light falls on a surface to create a 3D effect.
- Used to enhance the visualization of elevation data (e.g., DEMs).
- Generate hillshades using the 'Raster' > 'Analysis' > 'Hillshade' tool.
- Adjust parameters such as sun angle and azimuth for optimal visualization.



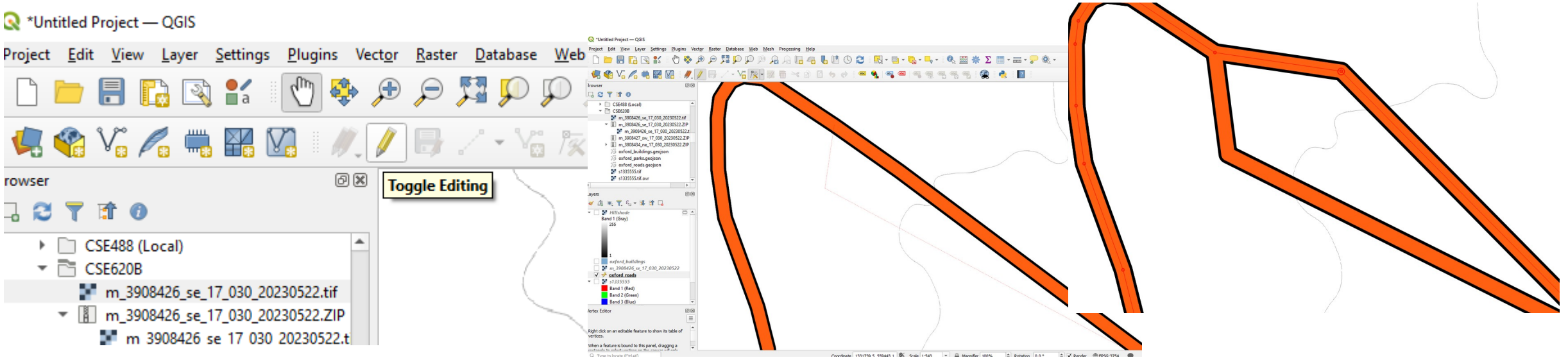
Styling Vector Data

- Style vector layers (e.g., shapefiles) to highlight features like roads, boundaries, or land use.
- Adjust symbology by changing color, line thickness, or fill patterns.
- Apply categorized or graduated styles to represent attribute data (e.g., population density, land cover type).



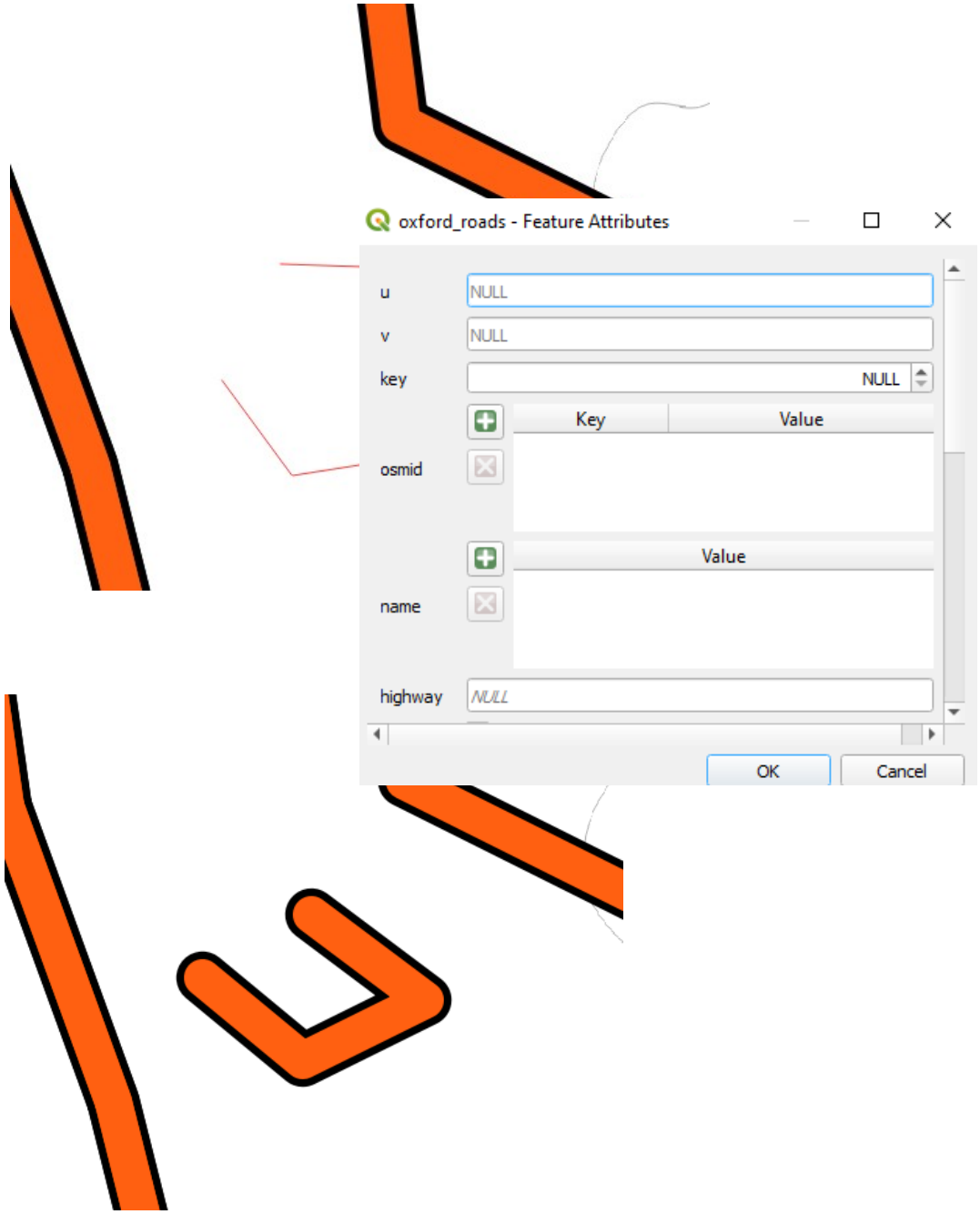
Editing Vector Layers in QGIS

- QGIS allows for interactive editing of vector data (points, lines, polygons).
- Useful for updating, cleaning, or refining data layers.
- Toggle editing mode for layers to make modifications.



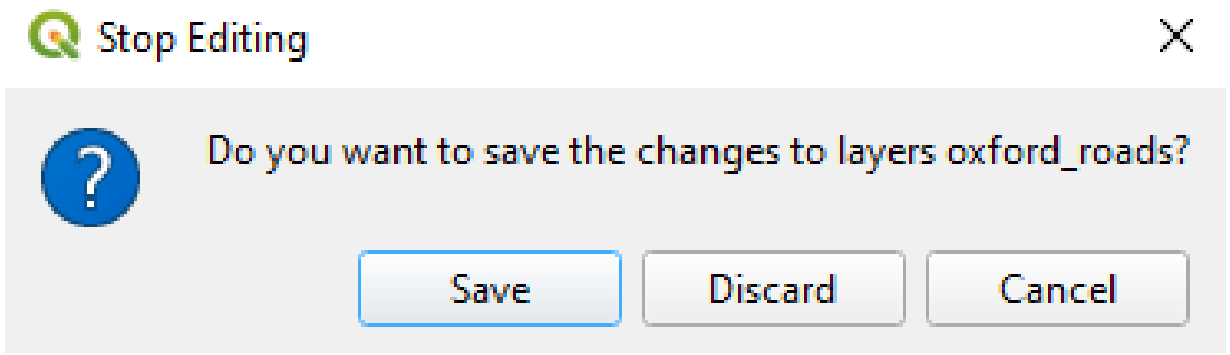
Adding New Features to a Vector Layer

- In editing mode, use the 'Add Feature' tool to create new points, lines, or polygons.
- Attribute information can be added or modified while creating new features.
- Snap features to existing ones for precision (e.g., aligning points along a boundary).



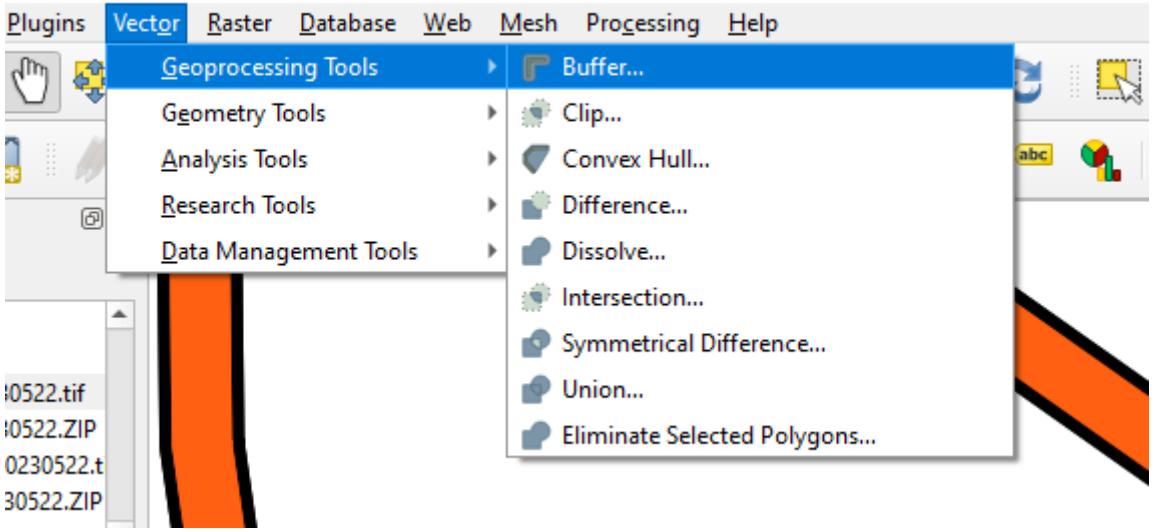
Saving and Committing Changes

- Once editing is done, save changes by committing the edits.
- Remember to save frequently to avoid losing work.



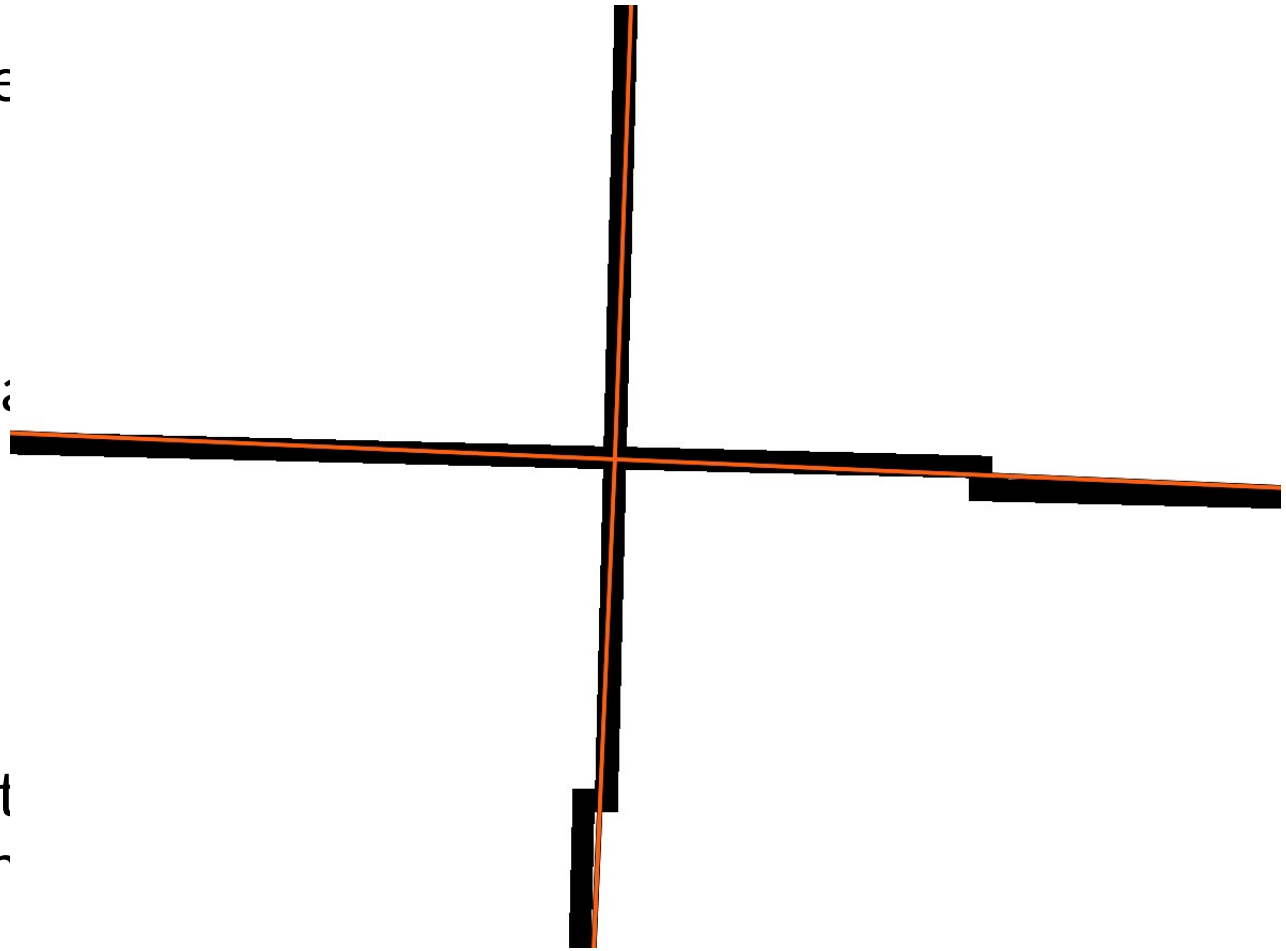
Other Useful Operations...

- **Buffer Analysis**: Create buffer zones around points, lines, or polygons for spatial analysis.
- **Intersect/Union**: Perform geospatial operations to combine or find common areas between layers.



Converting Vector Data to Raster in QGIS

- Useful for transforming vector feature (points, lines, polygons) into raster format.
- Often required for raster-based analysis like terrain modeling or spati analysis.
- Use the 'Rasterize (vector to raster)' tool found under 'Raster' > 'Conversion'.
- Choose an attribute to burn values int the raster, and set the pixel resolutior



Generating Distance Maps in QGIS

- A distance map calculates the distance of each raster cell to the nearest vector feature (e.g., roads, points of interest).
- Convert the vector layer to raster ‘Raster’>’Conversion’>’Rasterizes (Vector to Raster)’
- Use the 'Proximity (Raster Distance)' tool under 'Raster' > 'Analysis' to create a distance map.
- Distance maps are useful for proximity analysis in urban planning, conservation, and other spatial tasks.